

**Практика для цифровой кафедры СПбПУ
в формате
кейс-чемпионата
«ИТ-перспектива»**

Оглавление

<i>Роли в команде и их задачи</i>	3
<i>Описание кейсов</i>	4
I. Разработка ПО	4
1 кейс. "Умный город": Система отслеживания городских проблем	4
2 кейс. "Система управления складом": Учет и контроль товаров	8
3 кейс. "Образовательная платформа": Система онлайн-обучения	12
4 кейс. "Система мониторинга ИТ-инфраструктуры": Контроль состояния сетевых устройств	16
5 кейс. "Чат-бот для клиентской поддержки": Автоматизация обработки запросов	21
II. Анализ данных и машинное обучение	25
6 кейс. "Прогнозирование оттока клиентов банка"	25
7 кейс. "Анализ настроений в отзывах о продуктах"	27
8 кейс. "Прогнозирование цен на недвижимость"	29
9 кейс. "Сегментация клиентов интернет-магазина"	31
10 кейс. "Интерактивная аналитика больших данных"	33
III. Проектирование пользовательских интерфейсов	35
11 кейс. "Редизайн мобильного банкинга": Улучшение пользовательского опыта	35
12 кейс. "Дизайн-система для корпоративных продуктов": Создание единого визуального языка	39
13 кейс. "Интерфейс умного дома": Управление домашней автоматизацией	43
14 кейс. "Доступный интерфейс для людей с ограниченными возможностями": Инклюзивный дизайн цифровых сервисов	48
15 кейс. "Интерфейс аналитической платформы": Визуализация данных и бизнес-аналитика ..	53
IV. DevOps	58
16 кейс. "Создание системы автоматического резервного копирования для малого бизнеса": Защита данных предприятия	58
17 кейс. "Создание автоматизированного процесса развертывания простого веб-приложения": Основы CI/CD	62
18 кейс. "Разработка системы мониторинга и автоматического восстановления микросервисов": Обеспечение надежности приложений	66
19 кейс. "Разработка мультиоблачной платформы для управления микросервисами с системой самовосстановления": Современные облачные технологии	70

Роли в команде и их задачи

ИТ-роли:

1. Разработчик frontend

- Создание пользовательского интерфейса веб-приложения
- Реализация формы отправки сообщений о проблемах
- Интеграция с картографическим сервисом
- Разработка личного кабинета пользователя

2. Разработчик backend

- Создание базы данных для хранения информации о проблемах
- Разработка API для взаимодействия с фронтендом
- Реализация системы авторизации пользователей
- Создание механизма обработки загружаемых фотографий

3. Тестировщик

- Тестирование функциональности приложения
- Проверка корректности отображения данных на карте
- Выявление и документирование ошибок
- Тестирование адаптивности интерфейса

Не ИТ-роли:

4. Аналитик городских проблем

- Разработка классификации городских проблем
- Создание системы приоритетов для обращений
- Анализ собранных данных о проблемных зонах города
- Подготовка аналитических отчетов по типам проблем

5. Координатор взаимодействия с городскими службами

- Разработка регламента обработки поступающих сообщений
- Создание шаблонов ответов для различных типов обращений
- Разработка критериев для оценки решения проблем
- Планирование информационной кампании для привлечения пользователей

Описание кейсов

I. Разработка ПО

1 кейс. "Умный город": Система отслеживания городских проблем

Цель проекта: создать простое веб-приложение, позволяющее жителям города сообщать о проблемах городской инфраструктуры и отслеживать их решение.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА РАЗРАБОТКУ

1. Общие требования

1.1. Разработать веб-приложение с адаптивным дизайном (работа на компьютерах и мобильных устройствах).

1.2. Использовать простые технологии:

- Frontend: HTML, CSS, JavaScript (можно использовать Bootstrap для упрощения)
- Backend: на выбор команды (PHP, Python+Django, Node.js)
- База данных: SQLite или MySQL

1.3. Минимальные требования к производительности:

- Загрузка страниц не более 3 секунд
- Поддержка до 50 одновременных пользователей

2. Функциональные модули

2.1. Модуль регистрации и авторизации

2.1.1. Форма регистрации с полями:

- Имя пользователя
- Email
- Пароль
- Подтверждение пароля

2.1.2. Форма авторизации (логин/пароль)

2.1.3. Возможность восстановления пароля через email

2.2. Модуль отправки сообщений о проблемах

2.2.1. Форма создания заявки с полями:

- Тип проблемы (выпадающий список: яма на дороге, мусор, неработающее освещение, другое)
- Краткое описание (текстовое поле, до 200 символов)
- Подробное описание (текстовое поле, до 1000 символов)
- Адрес (текстовое поле с подсказками)
- Загрузка фото (до 3 фотографий, формат JPG/PNG, размер до 5 МБ каждая)

2.2.2. Автоматическое определение координат по введенному адресу

2.3. Модуль карты проблем

2.3.1. Интерактивная карта города (использовать бесплатные API, например OpenStreetMap)

2.3.2. Отображение маркеров проблем с цветовой индикацией:

- Красный — новая проблема
- Желтый — в обработке
- Зеленый — решена

2.3.3. При клике на маркер — отображение краткой информации о проблеме

2.3.4. Фильтрация проблем на карте:

- По типу проблемы
- По статусу
- По дате добавления

2.4. Личный кабинет пользователя

2.4.1. Список поданных пользователем заявок с информацией:

- Дата создания
- Тип проблемы
- Адрес
- Текущий статус
- Ожидаемая дата решения (если есть)

2.4.2. Возможность просмотра детальной информации по каждой заявке

2.4.3. Простая форма редактирования профиля пользователя

2.5. Административная панель

2.5.1. Доступ только для пользователей с правами администратора

2.5.2. Список всех заявок с возможностью:

- Просмотра деталей
- Изменения статуса
- Добавления комментария
- Назначения ожидаемой даты решения

2.5.3. Простая статистика:

- Количество заявок по типам
- Количество заявок по статусам
- Среднее время решения проблем

3. Требования к интерфейсу

3.1. Простой и понятный интерфейс с минимумом элементов

3.2. Использование готовых компонентов из библиотеки Bootstrap

3.3. Основные экраны:

- Главная страница с картой и краткой информацией о сервисе

- Страница создания заявки
- Личный кабинет
- Административная панель

4. Этапы разработки (для команды студентов)

4.1. Планирование (2-3 дня):

- Распределение ролей в команде
- Выбор технологий
- Создание простых макетов интерфейса
- Сформулировать требования к продукту
- Создать модели (в Archimate, bpmn, uml) – по желанию

4.2. Разработка базы данных (2-3 дня):

- Проектирование структуры таблиц
- Создание базы данных
- Заполнение тестовыми данными

4.3. Разработка backend (5-7 дней):

- Реализация авторизации
- Создание API для работы с заявками
- Интеграция с картографическим сервисом

4.4. Разработка frontend (5-7 дней):

- Верстка основных страниц
- Реализация интерактивных элементов
- Интеграция с backend

4.5. Тестирование и исправление ошибок (3-4 дня)

4.6. Подготовка презентации проекта (1-2 дня)

5. Критерии успешного выполнения

5.1. Работающее веб-приложение с реализацией всех основных функций

5.2. Понятный пользовательский интерфейс

5.3. Корректная работа на компьютере и мобильных устройствах

5.4. Отсутствие критических ошибок

5.5. Документация по проекту:

- Инструкция по установке
- Краткое руководство пользователя
- Описание архитектуры приложения

6. Дополнительные задания (по желанию)

6.1. Реализация уведомлений по email при изменении статуса заявки

- 6.2. Добавление рейтинга активности пользователей
- 6.3. Интеграция с социальными сетями для авторизации
- 6.4. Создание мобильного приложения (простой вариант)

2 кейс. "Система управления складом": Учет и контроль товаров

Цель проекта: создать простое веб-приложение для учета товаров на складе, позволяющее отслеживать поступления, отгрузки и текущие остатки.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА РАЗРАБОТКУ

1. Общие требования

1.1. Разработать веб-приложение с простым и понятным интерфейсом.

1.2. Использовать базовые технологии:

- Frontend: HTML, CSS, JavaScript (рекомендуется использовать Bootstrap)
- Backend: на выбор команды (PHP, Python+Flask, Node.js+Express)
- База данных: SQLite или MySQL

1.3. Минимальные требования к системе:

- Поддержка до 1000 наименований товаров
- Время отклика страниц не более 2 секунд
- Одновременная работа до 20 пользователей

2. Функциональные модули

2.1. Модуль авторизации

2.1.1. Простая форма входа:

- Логин
- Пароль

2.1.2. Два типа пользователей:

- Кладовщик (базовые операции)
- Администратор (полный доступ)

2.2. Модуль управления товарами

2.2.1. Форма добавления товара с полями:

- Наименование товара
- Категория (выбор из списка)
- Единица измерения
- Описание
- Место хранения (стеллаж/ячейка)
- Минимальный остаток для уведомления

2.2.2. Список товаров с возможностями:

- Просмотр всех товаров в табличном виде
- Редактирование информации о товаре
- Удаление товара (с подтверждением)

2.2.3. Управление категориями товаров:

- Добавление новых категорий
- Редактирование существующих
- Удаление категорий (если нет связанных товаров)

2.3. Модуль складских операций

2.3.1. Регистрация поступления товаров:

- Выбор товара из списка
- Указание количества
- Дата поступления
- Номер накладной
- Поставщик
- Комментарий (необязательно)

2.3.2. Регистрация отгрузки товаров:

- Выбор товара из списка
- Указание количества (не больше текущего остатка)
- Дата отгрузки
- Номер накладной
- Получатель
- Комментарий (необязательно)

2.3.3. Автоматическое обновление остатков после каждой операции

2.3.4. Журнал всех операций с возможностью фильтрации по дате и типу операции

2.4. Модуль поиска и фильтрации

2.4.1. Поиск товаров:

- По наименованию (частичное совпадение)
- По категории
- По месту хранения

2.4.2. Фильтрация списка товаров:

- По категории
- По наличию (все/в наличии/отсутствующие)
- По минимальному остатку (товары с критическим запасом)

2.5. Модуль отчетности

2.5.1. Формирование базовых отчетов:

- Текущие остатки всех товаров
- Товары с остатком ниже минимального
- Операции за выбранный период
- Движение конкретного товара за период

2.5.2. Экспорт отчетов:

- В формате CSV
- В формате PDF (опционально)

3. Требования к интерфейсу

3.1. Простой и интуитивно понятный интерфейс

3.2. Основные экраны:

- Главная страница (дашборд) с общей информацией и быстрыми ссылками
- Страница списка товаров
- Формы добавления/редактирования товаров
- Страница регистрации поступлений/отгрузок
- Журнал операций
- Страница отчетов

3.3. Адаптивный дизайн для работы на компьютерах и планшетах

4. Этапы разработки (для команды студентов)

4.1. Подготовка и планирование (2-3 дня):

- Распределение ролей в команде
- Выбор технологий
- Создание простых макетов интерфейса
- Сформулировать требования к продукту
- Создать модели (в Archimate, bpmn, uml) – по желанию

4.2. Проектирование базы данных (2-3 дня):

- Определение структуры таблиц
- Создание схемы базы данных
- Определение связей между таблицами

4.3. Разработка базовых функций (4-5 дней):

- Создание модуля авторизации
- Разработка модуля управления товарами
- Реализация базовых CRUD-операций

4.4. Разработка бизнес-логики (5-6 дней):

- Реализация складских операций
- Разработка системы поиска и фильтрации
- Создание модуля отчетности

4.5. Разработка интерфейса (4-5 дней):

- Верстка основных страниц
- Реализация интерактивных элементов
- Интеграция с бэкендом

4.6. Тестирование и отладка (3-4 дня):

- Проверка основных функций
- Исправление ошибок
- Оптимизация производительности

4.7. Подготовка документации и презентация (2 дня)

5. Критерии успешного выполнения

5.1. Работающее веб-приложение со всеми основными функциями

5.2. Корректная работа операций добавления, редактирования и удаления товаров

5.3. Правильный расчет остатков после операций поступления и отгрузки

5.4. Функционирующая система поиска и фильтрации

5.5. Возможность формирования и экспорта базовых отчетов

5.6. Документация по проекту:

- Руководство пользователя
- Инструкция по установке
- Описание структуры базы данных

6. Дополнительные задания (по желанию)

6.1. Реализация штрих-кодирования товаров

6.2. Добавление уведомлений о критических остатках

6.3. Создание более сложных отчетов с графиками и диаграммами

6.4. Реализация инвентаризации с формированием акта расхождений

3 кейс. "Образовательная платформа": Система онлайн-обучения

Цель проекта: создать простую образовательную веб-платформу, позволяющую размещать учебные материалы, проводить тестирование и отслеживать прогресс студентов.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА РАЗРАБОТКУ

1. Общие требования

1.1. Разработать веб-приложение с адаптивным дизайном для доступа с различных устройств.

1.2. Использовать базовые технологии:

- Frontend: HTML, CSS, JavaScript (рекомендуется Bootstrap)
- Backend: на выбор команды (PHP, Python+Django, Node.js)
- База данных: SQLite или MySQL

1.3. Минимальные требования к системе:

- Поддержка до 100 одновременных пользователей
- Время загрузки страниц не более 3 секунд
- Хранение до 500 МБ учебных материалов

2. Функциональные модули

2.1. Модуль регистрации и авторизации

2.1.1. Регистрация пользователей с полями:

- Имя и фамилия
- Email (используется как логин)
- Пароль
- Тип пользователя (студент/преподаватель)

2.1.2. Авторизация пользователей

2.1.3. Восстановление пароля через email

2.2. Модуль управления курсами

2.2.1. Для преподавателей:

- Создание нового курса с указанием названия и описания
- Добавление разделов и тем в курс
- Загрузка материалов (текст, PDF, изображения)
- Создание тестов для проверки знаний
- Публикация/скрытие курса

2.2.2. Для студентов:

- Просмотр доступных курсов
- Запись на курс
- Отмена записи на курс

2.3. Модуль учебных материалов

2.3.1. Типы учебных материалов:

- Текстовые страницы с форматированием
- PDF-документы
- Изображения
- Ссылки на внешние ресурсы (YouTube, статьи и т.д.)

2.3.2. Организация материалов:

- Разделение курса на разделы
- Разделение разделов на темы
- Последовательность изучения материалов

2.3.3. Отметка о прохождении материала студентом

2.4. Модуль тестирования

2.4.1. Создание тестов преподавателем:

- Вопросы с одним правильным ответом
- Вопросы с множественным выбором
- Вопросы с коротким текстовым ответом
- Настройка времени на прохождение теста
- Настройка проходного балла

2.4.2. Прохождение тестов студентами:

- Отображение вопросов и вариантов ответов
- Таймер обратного отсчета (если установлен)
- Отправка ответов на проверку

2.4.3. Автоматическая проверка тестов и выставление оценки

2.5. Модуль отслеживания прогресса

2.5.1. Для студентов:

- Отображение пройденных и непройденных материалов
- Результаты тестов
- Общий прогресс по курсу в процентах

2.5.2. Для преподавателей:

- Список студентов, записанных на курс
- Прогресс каждого студента
- Результаты тестирования
- Статистика по курсу (средний балл, процент завершения)

3. Требования к интерфейсу

3.1. Главная страница:

- Список доступных курсов
- Информация о платформе
- Панель навигации

3.2. Личный кабинет студента:

- Список курсов, на которые записан

- Прогресс по каждому курсу
- Результаты тестирования

3.3. Личный кабинет преподавателя:

- Список созданных курсов
- Инструменты для создания и редактирования материалов
- Статистика по курсам

3.4. Страница курса:

- Оглавление (разделы и темы)
- Область отображения материалов
- Кнопки навигации между материалами

3.5. Страница тестирования:

- Отображение вопроса и вариантов ответа
- Индикатор прогресса
- Кнопки навигации между вопросами

4. Этапы разработки (для команды студентов)

4.1. Планирование и анализ (2-3 дня):

- Распределение ролей в команде
- Выбор технологий
- Создание прототипов интерфейса
- Сформулировать требования к продукту
- Создать модели (в Archimate, bpmn, uml) – по желанию

4.2. Проектирование базы данных (2-3 дня):

- Определение структуры таблиц
- Создание схемы базы данных
- Определение связей между таблицами

4.3. Разработка модуля авторизации (3-4 дня):

- Реализация регистрации и входа
- Разграничение прав доступа
- Восстановление пароля

4.4. Разработка модуля курсов (4-5 дней):

- Создание, редактирование и удаление курсов
- Управление разделами и темами
- Запись студентов на курсы

4.5. Разработка модуля учебных материалов (4-5 дней):

- Загрузка и отображение материалов
- Организация последовательности изучения
- Отметка о прохождении

4.6. Разработка модуля тестирования (5-6 дней):

- Создание тестов преподавателем
- Прохождение тестов студентами
- Автоматическая проверка и оценка

4.7. Разработка модуля отслеживания прогресса (3-4 дня):

- Сбор данных о прогрессе студентов
- Отображение статистики
- Формирование отчетов

4.8. Тестирование и отладка (3-4 дня):

- Проверка всех функций
- Исправление ошибок
- Оптимизация работы

4.9. Подготовка документации и презентация (2 дня)

5. Критерии успешного выполнения

5.1. Работающая платформа со всеми основными функциями

5.2. Возможность создания курсов и добавления материалов

5.3. Функционирующая система тестирования

5.4. Корректное отслеживание прогресса студентов

5.5. Удобный и понятный интерфейс

5.6. Документация по проекту:

- Руководство пользователя для студентов
- Руководство пользователя для преподавателей
- Техническая документация по установке и настройке

6. Дополнительные задания (по желанию)

6.1. Реализация форума для обсуждения материалов курса

6.2. Добавление системы уведомлений (новые материалы, результаты тестов)

6.3. Интеграция с внешними образовательными ресурсами

6.4. Реализация системы выдачи сертификатов после завершения курса

6.5. Добавление элементов геймификации (баллы, рейтинги, достижения)

4 кейс. "Система мониторинга ИТ-инфраструктуры": Контроль состояния сетевых устройств

Цель проекта: создать простую систему мониторинга ИТ-инфраструктуры, позволяющую отслеживать состояние сетевых устройств, серверов и сервисов в режиме реального времени.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА РАЗРАБОТКУ

1. Общие требования

1.1. Разработать веб-приложение для мониторинга состояния ИТ-инфраструктуры.

1.2. Использовать базовые технологии:

- Frontend: HTML, CSS, JavaScript (рекомендуется Bootstrap)
- Backend: на выбор команды (Python+Flask, Node.js, PHP)
- База данных: SQLite или MySQL
- Протоколы мониторинга: ICMP (ping), SNMP (базовые метрики)

1.3. Минимальные требования к системе:

- Поддержка мониторинга до 50 устройств
- Интервал проверки от 1 минуты
- Хранение истории состояния устройств до 30 дней

2. Функциональные модули

2.1. Модуль авторизации

2.1.1. Простая форма входа:

- Логин
- Пароль

2.1.2. Два типа пользователей:

- Оператор (просмотр состояния)
- Администратор (полный доступ)

2.2. Модуль управления устройствами

2.2.1. Добавление нового устройства с параметрами:

- Название устройства
- IP-адрес или доменное имя
- Тип устройства (сервер, маршрутизатор, коммутатор и т.д.)
- Группа (для логического объединения устройств)
- Интервал проверки
- Методы мониторинга (ping, SNMP, проверка порта)

2.2.2. Редактирование параметров существующего устройства

2.2.3. Удаление устройства из системы мониторинга

2.2.4. Управление группами устройств:

- Создание новой группы
- Редактирование группы
- Удаление группы

2.3. Модуль мониторинга

2.3.1. Базовые методы проверки:

- Ping (доступность устройства)
- Проверка открытых портов (TCP/UDP)
- Базовый SNMP-мониторинг (загрузка CPU, использование памяти)

2.3.2. Автоматическое выполнение проверок с заданным интервалом

2.3.3. Сохранение результатов проверок в базу данных

2.3.4. Определение статуса устройства:

- Работает (зеленый)
- Предупреждение (желтый)
- Критическое состояние (красный)
- Недоступно (серый)

2.4. Модуль визуализации

2.4.1. Главная панель мониторинга (дашборд):

- Общее количество устройств по статусам
- Список устройств с критическими состояниями
- График доступности за последние 24 часа

2.4.2. Детальная информация по устройству:

- Текущий статус
- История статусов (график)
- Основные метрики (время отклика, загрузка CPU и т.д.)
- Журнал событий

2.4.3. Карта сети (простое визуальное представление):

- Отображение устройств в виде иконок
- Цветовая индикация статуса
- Группировка по типам или заданным группам

2.5. Модуль оповещений

2.5.1. Настройка правил оповещений:

- По типу устройства
- По группе устройств
- По типу события (недоступность, превышение порогов)

2.5.2. Способы оповещения:

- Отображение в интерфейсе
- Отправка email-уведомлений

2.5.3. Журнал оповещений с возможностью фильтрации

2.6. Модуль отчетности

2.6.1. Формирование базовых отчетов:

- Доступность устройств за период
- Статистика по времени отклика
- Количество инцидентов по устройствам

2.6.2. Экспорт отчетов в CSV-формат

3. Требования к интерфейсу

3.1. Главная страница (дашборд):

- Общая статистика по устройствам
- Список устройств с критическим статусом
- Графики основных метрик

3.2. Страница списка устройств:

- Табличное представление всех устройств
- Фильтрация по группам и статусам
- Быстрый доступ к детальной информации

3.3. Страница детальной информации по устройству:

- Графики метрик
- История статусов
- Журнал событий

3.4. Страница карты сети:

- Визуальное представление устройств
- Интерактивные элементы для получения информации

3.5. Страница настроек:

- Управление пользователями
- Настройка оповещений
- Общие параметры системы

4. Этапы разработки (для команды студентов)

4.1. Планирование и анализ (2-3 дня):

- Распределение ролей в команде
- Выбор технологий
- Определение архитектуры приложения
- Сформулировать требования к продукту
- Создать модели (в Archimate, bpmn, uml) – по желанию

4.2. Проектирование базы данных (2-3 дня):

- Определение структуры таблиц
- Создание схемы базы данных
- Определение связей между таблицами

4.3. Разработка модуля авторизации (2-3 дня):

- Реализация входа в систему
- Разграничение прав доступа

4.4. Разработка модуля управления устройствами (4-5 дней):

- Создание форм добавления и редактирования устройств
- Реализация группировки устройств
- Управление параметрами мониторинга

4.5. Разработка модуля мониторинга (5-6 дней):

- Реализация методов проверки (ping, порты, SNMP)
- Создание планировщика задач для регулярных проверок
- Сохранение результатов в базу данных

4.6. Разработка модуля визуализации (4-5 дней):

- Создание дашборда
- Реализация графиков и диаграмм
- Разработка простой карты сети

4.7. Разработка модуля оповещений (3-4 дня):

- Реализация механизма проверки условий
- Создание системы отправки уведомлений
- Ведение журнала оповещений

4.8. Разработка модуля отчетности (3-4 дня):

- Создание шаблонов отчетов
- Реализация экспорта данных

4.9. Тестирование и отладка (3-4 дня):

- Проверка всех функций
- Исправление ошибок
- Оптимизация производительности

4.10. Подготовка документации и презентация (2 дня)

5. Критерии успешного выполнения

5.1. Работающая система мониторинга с основными функциями

5.2. Возможность добавления и мониторинга устройств через ping и проверку портов

5.3. Корректное определение статуса устройств и отображение на дашборде

5.4. Функционирующая система оповещений о недоступности устройств

5.5. Возможность просмотра истории состояния устройств в виде графиков

5.6. Документация по проекту:

- Руководство пользователя
- Инструкция по установке и настройке
- Описание архитектуры системы

6. Дополнительные задания (по желанию)

6.1. Реализация более сложного SNMP-мониторинга (OID, MIB)

- 6.2. Добавление мониторинга веб-сервисов (HTTP, HTTPS)
- 6.3. Реализация автоматического обнаружения устройств в сети
- 6.4. Добавление интерактивной карты сети с визуализацией связей между устройствами
- 6.5. Реализация дополнительных каналов оповещения (Telegram, SMS)

5 кейс. "Чат-бот для клиентской поддержки": Автоматизация обработки запросов

Цель проекта: создать простой чат-бот для клиентской поддержки, способный отвечать на типовые вопросы пользователей и перенаправлять сложные запросы операторам.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА РАЗРАБОТКУ

1. Общие требования

1.1. Разработать чат-бот с веб-интерфейсом для взаимодействия с пользователями.

1.2. Использовать базовые технологии:

- Frontend: HTML, CSS, JavaScript (рекомендуется использование React или Vue.js)
- Backend: на выбор команды (Python+Flask, Node.js, PHP)
- База данных: SQLite или MongoDB
- NLP-библиотеки: простые решения (например, NLTK для Python)

1.3. Минимальные требования к системе:

- Поддержка до 50 одновременных диалогов
- Время ответа не более 2 секунд
- База знаний на 50-100 типовых вопросов

2. Функциональные модули

2.1. Модуль пользовательского интерфейса

2.1.1. Чат-виджет для веб-сайта:

- Всплывающее окно чата
- Поле ввода сообщения
- Область отображения истории диалога
- Индикатор состояния (бот отвечает, оператор подключен)

2.1.2. Стартовый экран:

- Приветственное сообщение
- Предложение выбрать категорию вопроса из меню
- Возможность ввести вопрос в свободной форме

2.2. Модуль обработки естественного языка

2.2.1. Базовый анализ запросов:

- Выделение ключевых слов
- Определение намерения пользователя
- Классификация запроса по категориям

2.2.2. Механизм поиска ответов:

- Сопоставление с базой знаний
- Ранжирование возможных ответов
- Выбор наиболее релевантного ответа

2.2.3. Обработка неопределенности:

- Запрос уточнений при неоднозначных вопросах
- Предложение вариантов при низкой уверенности в ответе

2.3. Модуль базы знаний

2.3.1. Структура базы знаний:

- Категории вопросов
- Вопросы с вариациями формулировок
- Ответы на вопросы
- Связанные вопросы и подсказки

2.3.2. Интерфейс управления базой знаний:

- Добавление новых вопросов и ответов
- Редактирование существующих записей
- Удаление устаревшей информации
- Импорт/экспорт базы знаний (CSV, JSON)

2.4. Модуль подключения оператора

2.4.1. Механизм эскалации:

- Автоматическое определение сложных вопросов
- Кнопка "Связаться с оператором" для пользователя
- Постановка запроса в очередь

2.4.2. Интерфейс оператора:

- Список активных диалогов
- История сообщений пользователя с ботом
- Поле для ответа пользователю
- Статусы диалогов (новый, в работе, завершен)

2.4.3. Передача управления:

- Уведомление пользователя о подключении оператора
- Возможность возврата диалога боту после решения проблемы

2.5. Модуль аналитики

2.5.1. Сбор статистики:

- Количество обработанных запросов
- Процент успешных ответов бота
- Частота обращений к оператору
- Популярные темы и вопросы

2.5.2. Отчеты и дашборды:

- Ежедневная статистика работы
- Анализ неотвеченных вопросов
- Время ответа и разрешения запросов

2.6. Модуль обучения бота

2.6.1. Механизм обратной связи:

- Оценка ответов пользователями (полезно/неполезно)
- Сбор неотвеченных вопросов

2.6.2. Простое обучение:

- Добавление новых вопросов на основе реальных запросов
- Корректировка ответов на основе обратной связи

3. Требования к интерфейсу

3.1. Пользовательский интерфейс:

- Минималистичный дизайн чат-виджета
- Адаптивность для мобильных устройств
- Индикация состояния (печатает, ожидание)
- Поддержка эмодзи и простого форматирования

3.2. Интерфейс администратора:

- Панель управления базой знаний
- Мониторинг активных диалогов
- Просмотр статистики и отчетов
- Настройка параметров работы бота

3.3. Интерфейс оператора:

- Список диалогов, требующих внимания
- Детальная информация о пользователе
- История взаимодействия с ботом
- Быстрые ответы на типовые вопросы

4. Этапы разработки (для команды студентов)

4.1. Планирование и анализ (2-3 дня):

- Распределение ролей в команде
- Выбор технологий
- Определение основных категорий вопросов
- Создание прототипов интерфейса
- Сформулировать требования к продукту
- Создать модели (в Archimate, bpmn, uml) – по желанию

4.2. Разработка базы знаний (3-4 дня):

- Составление списка типовых вопросов и ответов
- Структурирование по категориям
- Создание вариаций формулировок вопросов

4.3. Разработка модуля обработки естественного языка (5-6 дней):

- Реализация анализа текста
- Создание алгоритма поиска ответов
- Тестирование на типовых запросах

4.4. Разработка пользовательского интерфейса (4-5 дней):

- Создание чат-виджета
- Реализация взаимодействия с пользователем
- Интеграция с backend-частью

4.5. Разработка модуля подключения оператора (3-4 дня):

- Создание интерфейса оператора

- Реализация механизма эскалации
- Система очередей запросов

4.6. Разработка модуля аналитики (3-4 дня):

- Реализация сбора статистики
- Создание простых отчетов
- Визуализация данных

4.7. Разработка модуля обучения (2-3 дня):

- Реализация механизма обратной связи
- Создание инструментов для улучшения базы знаний

4.8. Тестирование и отладка (4-5 дней):

- Проверка работы бота на различных сценариях
- Исправление ошибок
- Оптимизация работы

4.9. Подготовка документации и презентация (2 дня)

5. Критерии успешного выполнения

5.1. Работающий чат-бот, способный отвечать на базовые вопросы из заданных категорий

5.2. Корректное определение намерения пользователя и классификация запросов

5.3. Функционирующий механизм эскалации запросов оператору

5.4. Удобный интерфейс для пользователей, операторов и администраторов

5.5. Возможность расширения базы знаний без программирования

5.6. Документация по проекту:

- Руководство пользователя
- Руководство оператора
- Руководство администратора
- Техническая документация по архитектуре

6. Дополнительные задания (по желанию)

6.1. Реализация мультязычной поддержки (русский и английский языки)

6.2. Добавление голосового ввода и вывода

6.3. Интеграция с популярными мессенджерами (Telegram, WhatsApp)

6.4. Реализация простых сценариев (последовательность вопросов для решения типовых задач)

6.5. Добавление персонализации (приветствие по имени, история обращений)

II. Анализ данных и машинное обучение

6 кейс. "Прогнозирование оттока клиентов банка"

Краткое описание функциональности

Проект "Прогнозирование оттока клиентов банка" представляет собой аналитическое приложение для анализа и прогнозирования оттока клиентов банка

Основная функциональность включает:

1. Интерактивные отчеты (дашборды) — инфографика по отчетам, которая показывает актуальную и объективную информацию для закрытия задач по принятию решений и контролю показателей бизнеса, проведения клиентской аналитики. Благодаря этой сведенной информации можно улучшить бизнес-процессы в банке и прогнозировать его развитие.

Создавать интерактивные отчеты (дашборды) можно в любой платформе бизнес-аналитики (например, Power BI, Tableau, Apache Superset или любой другой на выбор).

2. Разработанное аналитическое приложение прогнозной модели с использованием одной из библиотек (Gradio, Streamlit, Flask и любой другой на выбор) или просто в Colab (Jupyter Notebook) с анализом прогнозов, полученных с помощью обученной модели. Интерактивные визуализации, дашборды могут быть интегрированы в Colab (Kaggle Notebooks, Jupyter Notebook).

3. Сценарий (workflow) анализа данных, созданный в low-code платформе Loginom или Knime.

Роли в команде и их задачи

ИТ-роли:

1. Разработчик прикладного ПО на языке Python (Data Scientist)
- Исследовательский анализ данных, визуализация и машинное обучение: программирование на Python для анализа данных: Jupyter Notebook (Google Colab), NumPy, Pandas, Matplotlib, Seaborn, scikit-learn, Keras и др. (любые на выбор) фреймворки ML и/или Deep Learning;

- Создание веб-интерфейса для моделей машинного обучения с использованием фреймворков gradio/Streamlit/Flask (не обязательно)

2. Аналитик данных

- Создание интерактивных дашбордов, сторителлинг: поиск и объяснение инсайтов (BI платформы: Power BI Desktop, Tableau Public, Yandex DataLens, Apache Superset или любые другие на выбор)

Не ИТ-роли:

4. Продуктовый (маркетинговый, финансовый аналитик)

- Применение low-code инструмент для самостоятельного анализа
- Автоматизация рутинной работы, самостоятельный анализ данных без ожидания ответа от IT-департамента
- Принятие решений, основанные на данных

Примерный план работы на месяц

Неделя 1: Сбор и подготовка данных, исследовательский анализ данных (EDA).

Неделя 2: Моделирование и анализ, интерпретация результатов.

Неделя 3: Деплой и мониторинг.

Неделя 4: Итерации и улучшения, подготовка презентации проекта

7 кейс. "Анализ настроений в отзывах о продуктах"

Краткое описание функциональности

Проект "Анализ настроений в отзывах о продуктах" представляет собой аналитическое приложение для изучения отзывов клиентов маркетплейса

Основная функциональность включает:

1. Интерактивные отчеты (дашборды) для отслеживания и анализа основных метрик, визуализация данных, прогнозирования и моделирование, а также идентификации возможностей для улучшения бизнес-процессов.
Создавать интерактивные отчеты (дашборды) можно в любой платформе бизнес-аналитики (например, Power BI, Tableau, Apache Superset или любой другой на выбор).
2. Исследовательский анализ данных, в том числе с помощью визуализаций в Colab (Kaggle Notebooks, Jupyter Notebook). Разработка системы предоставления персонализированных, контекстно-зависимых рекомендаций.
3. Сценарий (workflow) анализа данных, созданный в low-code платформе Loginom или Knime.

Роли в команде и их задачи

ИТ-роли:

1. Специалист по работе с базами данных
 - Использование SQL для работы с базами данных и анализа данных (запросы на классическом языке SQL, и/или диалект PostgreSQL)
2. Data Scientist (Разработчик прикладного ПО на языке Python)
 - Исследовательский анализ данных, визуализация и машинное обучение: программирование на Python для анализа данных: Jupyter Notebook (Google Colab), NumPy, Pandas, Matplotlib, Seaborn, scikit-learn, Keras и др. (любые на выбор) фреймворки ML и/или Deep Learning;
 - Создание рекомендательной системы;
 - Создание веб-интерфейса для моделей машинного обучения с использованием фреймворков gradio/Streamlit/Flask (не обязательно)
3. Аналитик данных
 - Исследовательский анализ данных с использованием библиотек python
 - Создание интерактивных дашбордов, сторителлинг: поиск и объяснение инсайтов (BI платформы: Power BI Desktop, Tableau Public, Yandex DataLens, Apache Superset или любые другие на выбор)

Не ИТ-роли:

4. Маркетолог, продуктовый аналитик (бизнес-пользователь)
 - Применение low-code инструмент для самостоятельного анализа
 - Автоматизация рутинной работы, самостоятельный анализ данных без ожидания ответа от IT-департамента
 - Принятие решений, основанные на данных

Примерный план работы на месяц

Неделя 1: Исследовательский анализ данных (EDA).

Неделя 2: Разработка рекомендательной системы.

Неделя 3: Разработка дашбордов. Интеграция дашбордов в Colab и объединение с аналитикой на python и SQL.

Неделя 4: Итерации и улучшения, подготовка презентации проекта

8 кейс. "Прогнозирование цен на недвижимость"

Краткое описание функциональности

Проект "Прогнозирование цен на недвижимость" представляет собой аналитическое приложение для анализа зарегистрированных продаж недвижимости, динамики цен на жилье и прогнозирования цен на жилье

Основная функциональность включает:

1. Дашборды — наборы виджетов, которые можно группировать в удобном порядке, настраивать размер и добавлять пояснения. Дашборды позволяют следить за изменением метрик в реальном времени или анализировать накопленные метрики в динамике. Их целью которых является проведение оценки недвижимости и инвестиционных возможностей в секторе недвижимости, анализ динамики цен.

Создавать интерактивные отчеты (дашборды) можно в любой платформе бизнес-аналитики (например, Power BI, Tableau, Apache Superset или любой другой на выбор).

2. Исследовательский анализ данных, в том числе с помощью визуализаций в Colab (Kaggle Notebooks, Jupyter Notebook). Создание, обучение и объяснение модели прогнозирования цен на недвижимость в Colab (Jupyter Notebook). Интерактивные визуализации, дашборды могут быть также интегрированы в Colab (Kaggle Notebooks, Jupyter Notebook).

3. Сценарий (workflow) анализа данных, созданный в low-code платформе Loginom или Knime.

Роли в команде и их задачи

ИТ-роли:

1. Специалист по работе с базами данных

- Использование SQL для работы с базами данных и анализа данных (запросы на классическом языке SQL, и/или диалект PostgreSQL)

2. Data Scientist (Разработчик прикладного ПО на языке Python)

- Исследовательский анализ данных, визуализация и машинное обучение: программирование на Python для анализа данных: Jupyter Notebook (Google Colab), NumPy, Pandas, Matplotlib, Seaborn, scikit-learn, Keras и др. (любые на выбор) фреймворки ML и/или Deep Learning;

- Создание веб-интерфейса для моделей машинного обучения с использованием фреймворков gradio/Streamlit/Flask (не обязательно)

3. Аналитик данных

- Исследовательский анализ данных с использованием библиотек python
- Создание интерактивных дашбордов, сторителлинг: поиск и объяснение инсайтов (BI платформы: Power BI Desktop, Tableau Public, Yandex DataLens, Apache Superset или любые другие на выбор)

Не ИТ-роли:

4. Маркетолог, продуктовый аналитики (бизнес-пользователь)

- Применение low-code инструмент для самостоятельного анализа
- Автоматизация рутинной работы, самостоятельный данных без ожидания ответа от ИТ-департамента
- Принятие решений, основанные на данных

Примерный план работы на месяц

Неделя 1: Исследовательский анализ данных (EDA).

Неделя 2: Создание, обучение и объяснение модели.

Неделя 3: Разработка дашбордов. Интеграция дашбордов в Colab и объединение с аналитикой на python и SQL.

Неделя 4: Итерации и улучшения, подготовка презентации проекта

9 кейс. "Сегментация клиентов интернет-магазина"

Краткое описание функциональности

Проект "Сегментация клиентов интернет-магазина " представляет собой аналитическое приложение прогнозной модели (-ей) оттока клиентов и интерактивные дашборды, позволяющие провести анализ розничных продаж товаров и клиентскую аналитику.

Основная функциональность включает:

1. Интерактивные дашборды, позволяющие провести анализ розничных продаж товаров, клиентскую аналитику; проведенный сторителлинг (найденные инсайты). Благодаря этой сведенной информации можно улучшить бизнес-процессы в электронной торговле.

Создавать интерактивные отчеты (дашборды) можно в любой платформе бизнес-аналитики (например, Power BI, Tableau, Apache Superset или любой другой на выбор).

2. Исследовательский анализ данных, в том числе с помощью визуализаций в Colab (Kaggle Notebooks, Jupyter Notebook). Создание, обучение и объяснение модели оттока клиентов, которая позволит своевременно выявлять клиентов, которые могут перестать пользоваться услугами компании в ближайшем будущем.

Интерактивные визуализации, дашборды могут быть также интегрированы в Colab (Kaggle Notebooks, Jupyter Notebook).

3. Сценарий (workflow) анализа данных, созданный в low-code платформе Loginom или Knime.

Роли в команде и их задачи

ИТ-роли:

1. Специалист по работе с базами данных

- Использование SQL для работы с базами данных и анализа данных (запросы на классическом языке SQL, и/или диалект PostgreSQL)

2. Data Scientist (Разработчик прикладного ПО на языке Python)

- Исследовательский анализ данных, визуализация и машинное обучение: программирование на Python для анализа данных: Jupyter Notebook (Google Colab), NumPy, Pandas, Matplotlib, Seaborn, scikit-learn, Keras и др. (любые на выбор) фреймворки ML и/или Deep Learning;

- Создание веб-интерфейса для моделей машинного обучения с использованием фреймворков gradio/Streamlit/Flask (не обязательно)

3. Аналитик данных

- Исследовательский анализ данных с использованием библиотек python
- Создание интерактивных дашбордов, сторителлинг: поиск и объяснение инсайтов (BI платформы: Power BI Desktop, Tableau Public, Yandex DataLens, Apache Superset или любые другие на выбор)

Не ИТ-роли:

4. Маркетолог, продуктовый аналитики (бизнес-пользователь)

- Применение low-code инструмент для самостоятельного анализа
- Автоматизация рутинной работы, самостоятельный данных без ожидания ответа от ИТ-департамента
- Принятие решений, основанные на данных

Примерный план работы на месяц

Неделя 1: Сбор и подготовка данных, исследовательский анализ данных (EDA).

Неделя 2: Создание, обучение и объяснение модели.

Неделя 3: Разработка дашбордов. Интеграция дашбордов в Colab и объединение с аналитикой на python и SQL.

Неделя 4: Итерации и улучшения, подготовка презентации проекта

10 кейс. "Интерактивная аналитика больших данных"

Краткое описание функциональности

Проект "Интерактивная аналитика больших данных" представляет собой аналитическое приложение анализа городской системы проката велосипедов и заказов такси.

Основная функциональность включает:

1. Интерактивные дашборды, позволяющие провести анализ, когда на велосипеде добраться до места назначения быстрее чем на такси, в зависимости от маршрута и времени суток.

Создавать интерактивные отчеты (дашборды) можно в любой платформе бизнес-аналитики (например, Power BI, Tableau, Apache Superset или любой другой на выбор).

2. Исследовательский анализ данных, в том числе с помощью визуализаций в Colab (BigQuery, Kaggle Notebooks, Jupyter Notebook) для разработки рекомендаций жителям города по оптимизации перемещения между различными районами в разное время суток. Интерактивные визуализации, дашборды могут быть также интегрированы в Colab (Kaggle Notebooks, Jupyter Notebook).

3. Сценарий (workflow) анализа данных, созданный в low-code платформе Loginom или Knime.

Роли в команде и их задачи

ИТ-роли:

1. Специалист по работе с базами данных

- Использование SQL для работы с базами данных и анализа данных (запросы на классическом языке SQL, и/или диалект PostgreSQL)

2. Data Scientist (Разработчик прикладного ПО на языке Python)

- Исследовательский анализ данных, визуализация и машинное обучение: программирование на Python для анализа данных: Jupyter Notebook (Google Colab), NumPy, Pandas, Matplotlib, Seaborn, scikit-learn, Keras и др. (любые на выбор) фреймворки ML и/или Deep Learning;

- Создание веб-интерфейса для моделей машинного обучения с использованием фреймворков gradio/Streamlit/Flask (не обязательно)

3. Аналитик данных

- Исследовательский анализ данных с использованием библиотек python
- Создание интерактивных дашбордов, сторителлинг: поиск и объяснение инсайтов (BI платформы: Power BI Desktop, Tableau Public, Yandex DataLens, Apache Superset или любые другие на выбор)

Не ИТ-роли:

4. Маркетолог, продуктовый аналитики (бизнес-пользователь)

- Применение low-code инструмент для самостоятельного анализа
- Автоматизация рутинной работы, самостоятельный анализ данных без ожидания ответа от IT-департамента
- Принятие решений, основанные на данных

Примерный план работы на месяц

Неделя 1: Сбор и подготовка данных.

Неделя 2: Исследовательский анализ данных (EDA).

Неделя 3: Разработка дашбордов. Интеграция дашбордов в Colab и объединение с аналитикой на python и SQL.

Неделя 4: Итерации и улучшения, подготовка презентации проекта

III. Проектирование пользовательских интерфейсов

11 кейс. "Редизайн мобильного банкинга": Улучшение пользовательского опыта

Цель проекта: разработать новый дизайн интерфейса мобильного банковского приложения, повышающий удобство использования и визуальную привлекательность.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПРОЕКТИРОВАНИЕ

1. Общие требования

1.1. Разработать дизайн-концепцию мобильного банковского приложения для платформ iOS и Android.

1.2. Рекомендуемые инструменты:

- Figma, Adobe XD или Sketch для создания прототипов
- Miro или FigJam для построения карты пользовательского пути
- Простые инструменты для проведения пользовательских исследований

1.3. Минимальные требования к дизайну:

- Соответствие современным трендам UI/UX дизайна
- Адаптивность для различных размеров экранов смартфонов
- Соблюдение принципов доступности (accessibility)

2. Функциональные модули для проектирования

2.1. Модуль авторизации

2.1.1. Экраны для проектирования:

- Стартовый экран
- Вход по логину и паролю
- Вход по биометрии (отпечаток пальца/Face ID)
- Восстановление доступа
- Регистрация нового пользователя

2.1.2. Требования к дизайну авторизации:

- Простой и понятный процесс
- Минимальное количество шагов
- Четкие инструкции для пользователя

2.2. Главный экран (дашборд)

2.2.1. Элементы для проектирования:

- Отображение баланса основного счета
- Список карт пользователя
- Быстрые действия (перевод, оплата, пополнение)
- Последние операции
- Персональные предложения

2.2.2. Требования к дизайну дашборда:

- Информативность без перегруженности

- Визуальная иерархия элементов
- Быстрый доступ к основным функциям

2.3. Модуль управления картами и счетами

2.3.1. Экраны для проектирования:

- Список всех карт и счетов
- Детальная информация по карте/счету
- Настройки карты (лимиты, блокировка)
- Выписка операций с фильтрацией
- Заказ новой карты

2.3.2. Требования к дизайну:

- Наглядное представление финансовой информации
- Удобная навигация между картами/счетами
- Понятная группировка операций

2.4. Модуль платежей и переводов

2.4.1. Экраны для проектирования:

- Перевод между своими счетами
- Перевод по номеру телефона/карты
- Оплата услуг (категории услуг)
- Оплата по QR-коду
- Подтверждение операции
- Результат операции (успех/ошибка)

2.4.2. Требования к дизайну:

- Минимальное количество шагов для совершения операции
- Понятная форма ввода данных
- Четкое отображение комиссий и деталей перевода

2.5. Модуль финансовой аналитики

2.5.1. Экраны для проектирования:

- Общая статистика расходов и доходов
- Распределение трат по категориям
- Динамика расходов (графики/диаграммы)
- Бюджетирование и цели

2.5.2. Требования к дизайну:

- Наглядная визуализация данных
- Интуитивно понятные графики и диаграммы
- Возможность настройки периода анализа

3. Требования к пользовательскому интерфейсу

3.1. Общие принципы дизайна:

- Единый стиль оформления всех экранов
- Понятная система навигации
- Минималистичный дизайн без лишних элементов
- Читаемые шрифты и контрастные цвета

3.2. Цветовая схема:

- Разработать основную цветовую палитру (3-5 основных цветов)
- Предусмотреть темную тему
- Учесть цветовую дифференциацию для различных типов операций

3.3. Типографика:

- Выбрать системные шрифты для максимальной совместимости
- Определить иерархию текста (заголовки, подзаголовки, основной текст)
- Обеспечить хорошую читаемость на разных размерах экрана

3.4. Компоненты интерфейса:

- Разработать библиотеку базовых UI-компонентов (кнопки, поля ввода, карточки)
- Создать варианты состояний для интерактивных элементов
- Предусмотреть анимации для ключевых взаимодействий

4. Этапы проектирования (для команды студентов)

4.1. Исследование и анализ (3-4 дня):

- Анализ существующих банковских приложений
- Выявление сильных и слабых сторон конкурентов
- Составление портретов пользователей (персон)
- Определение основных пользовательских сценариев
- Сформулировать требования к продукту
- Создать модели (в Archimate, bpmn, uml) – по желанию

4.2. Проектирование информационной архитектуры (2-3 дня):

- Создание карты приложения
- Определение основных разделов и их взаимосвязей
- Разработка пользовательских путей

4.3. Создание вайрфреймов (4-5 дней):

- Разработка низкодетализированных макетов основных экранов
- Определение расположения основных элементов
- Проработка навигации между экранами

4.4. Разработка дизайн-концепции (3-4 дня):

- Создание мудборда (коллекции визуальных референсов)
- Определение стилистики, цветовой схемы и типографики
- Разработка ключевых экранов в выбранном стиле

4.5. Создание прототипов (5-6 дней):

- Детальная проработка всех экранов
- Создание интерактивных связей между экранами
- Добавление микроанимаций и переходов

4.6. Пользовательское тестирование (2-3 дня):

- Подготовка сценариев для тестирования
- Проведение тестирования на целевой аудитории
- Сбор и анализ обратной связи

4.7. Доработка дизайна (3-4 дня):

- Внесение изменений на основе результатов тестирования
- Финализация дизайн-системы
- Подготовка спецификаций для разработчиков

4.8. Подготовка презентации проекта (2 дня)

5. Критерии успешного выполнения

- 5.1. Полный набор спроектированных экранов для всех основных функций приложения
- 5.2. Логичная и понятная информационная архитектура
- 5.3. Единый визуальный стиль всех элементов интерфейса
- 5.4. Интерактивный прототип, демонстрирующий основные пользовательские сценарии
- 5.5. Обоснование принятых дизайнерских решений
- 5.6. Документация по проекту:
 - Описание целевой аудитории
 - Пользовательские сценарии
 - Дизайн-система (цвета, шрифты, компоненты)
 - Рекомендации по реализации

6. Дополнительные задания (по желанию)

- 6.1. Разработка анимированных прототипов для ключевых взаимодействий
- 6.2. Создание дизайна для дополнительных функций (инвестиции, кредиты, страхование)
- 6.3. Проработка системы персонализации интерфейса
- 6.4. Разработка версии для планшетов
- 6.5. Создание маркетинговых материалов для продвижения обновленного приложения

12 кейс. "Дизайн-система для корпоративных продуктов": Создание единого визуального языка

Цель проекта: разработать базовую дизайн-систему для линейки корпоративных продуктов, обеспечивающую единый визуальный стиль и ускоряющую процесс создания новых интерфейсов.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПРОЕКТИРОВАНИЕ

1. Общие требования

1.1. Разработать дизайн-систему для корпоративных веб-приложений, ориентированных на бизнес-пользователей.

1.2. Рекомендуемые инструменты:

- Figma или Adobe XD для создания компонентов и документации
- Storybook для демонстрации интерактивных компонентов (опционально)
- Miro или FigJam для коллективной работы над структурой

1.3. Минимальные требования к дизайн-системе:

- Масштабируемость и гибкость для применения в различных продуктах
- Соответствие принципам доступности (WCAG 2.1 уровень AA)
- Адаптивность для различных размеров экранов (от 1024px)

2. Компоненты дизайн-системы

2.1. Основы дизайн-системы

2.1.1. Дизайн-токены:

- Цветовая палитра (основные, акцентные, семантические цвета)
- Типографика (шрифты, размеры, интерлиньяж)
- Отступы и размеры (система сеток и интервалов)
- Тени и эффекты
- Скругления углов

2.1.2. Принципы и правила:

- Основные принципы дизайна продуктов
- Правила использования компонентов
- Рекомендации по композиции интерфейсов

2.2. Базовые компоненты

2.2.1. Компоненты ввода:

- Текстовые поля (однострочные, многострочные)
- Чекбоксы и радиокнопки
- Переключатели (toggle)
- Селекты и дропдауны
- Слайдеры и спиннеры

2.2.2. Компоненты действий:

- Кнопки (первичные, вторичные, третичные)
- Иконки (набор из 20-30 основных иконок)

- Ссылки
- Меню и контекстные меню

2.2.3. Компоненты отображения информации:

- Бейджи и метки
- Уведомления и алерты
- Прогресс-бары и индикаторы загрузки
- Тултипы и подсказки
- Пагинация

2.3. Составные компоненты

2.3.1. Навигационные компоненты:

- Верхняя панель навигации
- Боковое меню
- Хлебные крошки
- Табы и вкладки
- Степперы (пошаговые процессы)

2.3.2. Компоненты для работы с данными:

- Таблицы с сортировкой и фильтрацией
- Карточки и списки
- Простые графики и диаграммы
- Формы ввода данных

2.3.3. Модальные компоненты:

- Диалоговые окна
- Модальные формы
- Сайдбары и панели
- Всплывающие подсказки

2.4. Шаблоны страниц

2.4.1. Типовые шаблоны:

- Страница входа и регистрации
- Дашборд с виджетами
- Страница списка элементов
- Страница детального просмотра
- Страница настроек
- Страница с формой

2.4.2. Структурные элементы:

- Хедер и футер
- Сайдбары
- Основные типы контентных блоков
- Системные сообщения

3. Требования к документации

3.1. Общая структура документации:

- Введение в дизайн-систему
- Принципы и философия дизайна

- Руководство по использованию

3.2. Документация по компонентам:

- Описание назначения компонента
- Варианты и состояния
- Примеры использования
- Правила и ограничения

3.3. Рекомендации по применению:

- Примеры композиции интерфейсов
- Решение типовых задач
- Антипаттерны (чего следует избегать)

4. Этапы проектирования (для команды студентов)

4.1. Исследование и анализ (3-4 дня):

- Изучение существующих дизайн-систем (Material Design, IBM Carbon, Atlassian)
- Анализ потребностей корпоративных пользователей
- Определение ключевых сценариев использования
- Формирование требований к компонентам
- Сформулировать требования к продукту
- Создать модели (в Archimate, bpmn, uml) – по желанию

4.2. Разработка основ дизайн-системы (3-4 дня):

- Создание цветовой палитры
- Определение типографической системы
- Разработка системы отступов и размеров
- Формулирование основных принципов

4.3. Проектирование базовых компонентов (5-6 дней):

- Создание эскизов компонентов
- Разработка различных состояний (default, hover, focus, disabled)
- Создание компонентов в Figma с использованием Auto Layout
- Организация компонентов в библиотеку

4.4. Проектирование составных компонентов (5-6 дней):

- Создание более сложных компонентов на основе базовых
- Разработка вариантов и модификаций
- Проверка компонентов на соответствие принципам доступности
- Добавление компонентов в библиотеку

4.5. Создание шаблонов страниц (4-5 дней):

- Разработка типовых макетов страниц
- Демонстрация применения компонентов в контексте
- Проверка удобства использования

4.6. Подготовка документации (4-5 дней):

- Описание принципов дизайн-системы
- Создание руководства по использованию компонентов
- Разработка примеров и рекомендаций

4.7. Тестирование и доработка (3-4 дня):

- Проверка консистентности компонентов
- Тестирование на различных размерах экранов
- Внесение корректировок на основе обратной связи

4.8. Подготовка презентации проекта (2 дня)

5. Критерии успешного выполнения

5.1. Разработанная библиотека компонентов в Figma или Adobe XD

5.2. Полный набор дизайн-токенов (цвета, типографика, отступы)

5.3. Минимум 15-20 базовых компонентов с различными состояниями

5.4. Минимум 10 составных компонентов

5.5. 5-6 шаблонов типовых страниц

5.6. Документация по использованию дизайн-системы

6. Дополнительные задания (по желанию)

6.1. Создание темной темы для всех компонентов

6.2. Разработка анимаций и переходов для интерактивных элементов

6.3. Создание версии компонентов для мобильных устройств

6.4. Прототипирование компонентов в коде (HTML/CSS)

6.5. Разработка плагина или расширения для дизайн-системы

13 кейс. "Интерфейс умного дома": Управление домашней автоматизацией

Цель проекта: разработать интуитивно понятный пользовательский интерфейс для системы умного дома, позволяющий эффективно управлять различными домашними устройствами и сценариями.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПРОЕКТИРОВАНИЕ

1. Общие требования

1.1. Разработать дизайн мобильного приложения для управления системой умного дома с возможностью адаптации для планшетов.

1.2. Рекомендуемые инструменты:

- Figma или Adobe XD для создания макетов и прототипов
- Miro или FigJam для построения пользовательских сценариев
- Простые инструменты для проведения пользовательских тестов

1.3. Минимальные требования к дизайну:

- Интуитивно понятный интерфейс, доступный пользователям разных возрастов
- Поддержка светлой и темной темы
- Соответствие принципам доступности (крупные элементы управления, контрастные цвета)

2. Функциональные модули для проектирования

2.1. Модуль авторизации и настройки

2.1.1. Экраны для проектирования:

- Вход в приложение (логин/пароль, биометрия)
- Регистрация нового пользователя
- Добавление нового дома/квартиры
- Настройка профиля пользователя
- Управление доступом для членов семьи

2.1.2. Требования к дизайну:

- Простой процесс первоначальной настройки
- Понятная форма добавления новых устройств
- Визуальное разделение между домами (для пользователей с несколькими объектами)

2.2. Главный экран (дашборд)

2.2.1. Элементы для проектирования:

- Общий статус системы умного дома
- Быстрый доступ к часто используемым устройствам
- Активные сценарии
- Уведомления и предупреждения
- Погода и климат в доме

2.2.2. Требования к дизайну:

- Наглядное представление состояния основных систем
- Возможность быстрого управления без перехода в другие разделы
- Персонализация расположения элементов

2.3. Модуль управления помещениями

2.3.1. Экраны для проектирования:

- Список комнат/зон дома
- Детальный вид комнаты со всеми устройствами
- Добавление новой комнаты
- Настройка параметров помещения

2.3.2. Требования к дизайну:

- Визуальное представление плана дома/квартиры
- Интуитивная навигация между помещениями
- Группировка устройств по типам внутри помещения

2.4. Модуль управления устройствами

2.4.1. Типы устройств для проектирования:

- Освещение (включение/выключение, яркость, цвет)
- Климат-контроль (термостаты, кондиционеры)
- Безопасность (камеры, датчики движения, замки)
- Бытовая техника (телевизоры, аудиосистемы, кухонные приборы)
- Шторы и жалюзи

2.4.2. Экраны для проектирования:

- Список всех устройств по категориям
- Детальное управление отдельным устройством
- Добавление нового устройства
- Настройка параметров устройства

2.4.3. Требования к дизайну:

- Унифицированный стиль управления разными типами устройств
- Наглядное отображение текущего состояния
- Интуитивные элементы управления (слайдеры, переключатели)

2.5. Модуль сценариев и автоматизации

2.5.1. Экраны для проектирования:

- Список доступных сценариев
- Создание нового сценария
- Настройка условий и действий
- Расписание автоматизаций

2.5.2. Требования к дизайну:

- Визуальный конструктор сценариев (если/то)
- Понятное представление цепочки действий
- Временная шкала для настройки расписаний

2.6. Модуль статистики и аналитики

2.6.1. Экраны для проектирования:

- Общий обзор энергопотребления
- Детальная статистика по устройствам
- Графики использования систем
- История событий и активности

2.6.2. Требования к дизайну:

- Наглядные графики и диаграммы
- Фильтры по периодам и типам данных
- Рекомендации по оптимизации использования

3. Требования к пользовательскому интерфейсу

3.1. Общие принципы дизайна:

- Минималистичный и современный стиль
- Акцент на визуальной информации (иконки, цветовые индикаторы)
- Тактильная обратная связь для основных действий
- Единообразие элементов управления

3.2. Цветовая схема:

- Основная палитра из 3-5 цветов
- Семантическое использование цветов (зеленый - включено, красный - проблема)
- Высокий контраст для важных элементов управления
- Адаптация цветов для темной темы

3.3. Типографика:

- Четкая иерархия текста
- Легко читаемые шрифты
- Достаточный размер текста для пользователей всех возрастов

3.4. Навигация:

- Понятная структура приложения
- Быстрый доступ к основным функциям
- Возможность возврата на предыдущий экран
- Поиск устройств и функций

4. Этапы проектирования (для команды студентов)

4.1. Исследование и анализ (3-4 дня):

- Изучение существующих решений для умного дома
- Анализ пользовательских потребностей
- Определение ключевых сценариев использования
- Создание портретов пользователей (персон)
- Сформулировать требования к продукту
- Создать модели (в Archimate, bpmn, uml) – по желанию

4.2. Проектирование информационной архитектуры (2-3 дня):

- Разработка структуры приложения
- Определение основных разделов
- Создание карты пользовательских путей
- Планирование навигации

4.3. Создание вайрфреймов (4-5 дней):

- Разработка низкодетализированных макетов основных экранов
- Проработка расположения элементов управления
- Определение взаимодействий между экранами

4.4. Разработка дизайн-концепции (3-4 дня):

- Определение визуального стиля
- Создание мудборда (коллекции визуальных референсов)
- Проработка цветовой схемы и типографики
- Дизайн ключевых экранов

4.5. Детальное проектирование интерфейса (5-6 дней):

- Создание всех необходимых экранов
- Проработка элементов управления для разных типов устройств
- Разработка иконок и визуальных элементов
- Создание светлой и темной темы

4.6. Прототипирование (3-4 дня):

- Создание интерактивного прототипа
- Настройка переходов между экранами
- Имитация основных функций приложения

4.7. Пользовательское тестирование (2-3 дня):

- Подготовка сценариев тестирования
- Проведение тестирования с потенциальными пользователями
- Сбор и анализ обратной связи

4.8. Доработка дизайна (3-4 дня):

- Внесение изменений на основе результатов тестирования
- Улучшение проблемных мест интерфейса
- Финализация дизайна

4.9. Подготовка презентации проекта (2 дня)

5. Критерии успешного выполнения

5.1. Полный набор спроектированных экранов для всех основных функций приложения

5.2. Интуитивно понятный интерфейс, подтвержденный результатами тестирования

5.3. Визуально привлекательный дизайн с единым стилем

5.4. Проработанные элементы управления для разных типов устройств

5.5. Интерактивный прототип, демонстрирующий основные пользовательские сценарии

5.6. Документация по проекту:

- Описание концепции интерфейса
- Пользовательские сценарии
- Руководство по стилю (цвета, шрифты, компоненты)
- Рекомендации по дальнейшему развитию

6. Дополнительные задания (по желанию)

6.1. Разработка голосового интерфейса управления

6.2. Создание версии для умных дисплеев (Amazon Echo Show, Google Nest Hub)

6.3. Проектирование виджетов для быстрого доступа с экрана блокировки

6.4. Разработка интерфейса для умных часов

6.5. Создание AR-режима для визуализации управления устройствами через камеру смартфона

14 кейс. "Доступный интерфейс для людей с ограниченными возможностями": Инклюзивный дизайн цифровых сервисов

Цель проекта: разработать инклюзивный дизайн веб-сервиса или мобильного приложения, обеспечивающий полноценный доступ пользователям с различными ограниченными возможностями.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПРОЕКТИРОВАНИЕ

1. Общие требования

1.1. Разработать дизайн информационного портала или сервиса (новостной сайт, образовательная платформа или государственный сервис) с учетом потребностей пользователей с ограниченными возможностями.

1.2. Рекомендуемые инструменты:

- Figma или Adobe XD для создания макетов и прототипов
- Инструменты для проверки контрастности (Contrast Checker)
- Симуляторы различных нарушений зрения (Color Oracle, NoCoffee)
- Средства проверки доступности (Axe, WAVE)

1.3. Минимальные требования к дизайну:

- Соответствие стандартам WCAG 2.1 уровня AA
- Адаптивность для различных устройств
- Совместимость с программами экранного доступа

2. Целевые группы пользователей

2.1. Пользователи с нарушениями зрения:

- Слабовидящие
- Пользователи с дальтонизмом
- Незрячие пользователи, использующие скринридеры

2.2. Пользователи с нарушениями моторики:

- Ограниченная подвижность рук
- Трудности с точными движениями
- Использующие альтернативные устройства ввода

2.3. Пользователи с нарушениями слуха:

- Слабослышащие
- Глухие пользователи

2.4. Пользователи с когнитивными особенностями:

- Трудности с концентрацией внимания
- Проблемы с восприятием сложной информации
- Дислексия и другие особенности обработки текста

3. Функциональные модули для проектирования

3.1. Модуль настройки доступности

3.1.1. Элементы для проектирования:

- Панель настроек доступности
- Переключение между режимами отображения
- Настройка размера текста и контрастности
- Включение/отключение анимаций
- Сохранение пользовательских настроек

3.1.2. Требования к дизайну:

- Легкодоступное расположение панели настроек
- Простые и понятные элементы управления
- Предварительный просмотр изменений

3.2. Главная страница и навигация

3.2.1. Элементы для проектирования:

- Структура главной страницы
- Главное меню навигации
- Поиск по сайту
- Хлебные крошки
- Карта сайта

3.2.2. Требования к дизайну:

- Четкая иерархия информации
- Несколько способов навигации
- Возможность пропуска повторяющихся блоков
- Понятные и информативные заголовки

3.3. Модуль представления контента

3.3.1. Элементы для проектирования:

- Текстовые блоки
- Изображения и мультимедиа
- Таблицы данных
- Списки и перечисления
- Интерактивные элементы

3.3.2. Требования к дизайну:

- Структурированное представление информации
- Альтернативные описания для нетекстового контента
- Возможность масштабирования без потери функциональности
- Четкое форматирование для улучшения читаемости

3.4. Модуль форм и ввода данных

3.4.1. Элементы для проектирования:

- Текстовые поля
- Чекбоксы и радиокнопки
- Выпадающие списки
- Загрузка файлов
- Кнопки отправки и сброса

3.4.2. Требования к дизайну:

- Четкая маркировка всех полей ввода
- Понятные инструкции и подсказки
- Доступная валидация и сообщения об ошибках
- Возможность навигации с клавиатуры

3.5. Модуль уведомлений и обратной связи

3.5.1. Элементы для проектирования:

- Системные уведомления
- Сообщения об ошибках
- Подтверждения действий
- Индикаторы загрузки и прогресса

3.5.2. Требования к дизайну:

- Мультимодальные уведомления (визуальные и текстовые)
- Достаточное время для восприятия информации
- Четкие инструкции по исправлению ошибок
- Возможность отмены действий

4. Требования к пользовательскому интерфейсу

4.1. Визуальное оформление:

- Высокий контраст между текстом и фоном (минимум 4.5:1)
- Достаточный размер шрифта (минимум 16px для основного текста)
- Четкая визуальная иерархия элементов
- Неиспользование цвета как единственного средства передачи информации

4.2. Типографика:

- Использование читабельных шрифтов без засечек
- Достаточный межстрочный интервал (минимум 1.5)
- Выравнивание текста по левому краю
- Ограничение длины строки (60-80 символов)

4.3. Интерактивные элементы:

- Крупные активные области для кликабельных элементов (минимум 44×44px)
- Четкое визуальное состояние фокуса для навигации с клавиатуры
- Понятные метки и иконки с текстовыми подписями
- Достаточное расстояние между интерактивными элементами

4.4. Мультимедиа:

- Субтитры для видеоконтента
- Текстовые расшифровки для аудио
- Возможность управления воспроизведением
- Отсутствие автоматического воспроизведения

5. Этапы проектирования (для команды студентов)

5.1. Исследование и анализ (3-4 дня):

- Изучение стандартов доступности (WCAG 2.1)
- Исследование потребностей различных групп пользователей
- Анализ существующих решений и лучших практик
- Определение ключевых сценариев использования

- Сформулировать требования к продукту
- Создать модели (в Archimate, bpmn, uml) – по желанию

5.2. Определение концепции (2-3 дня):

- Выбор типа сервиса для проектирования
- Определение основных функций и содержания
- Создание карты пользовательских путей
- Разработка информационной архитектуры

5.3. Создание базовых компонентов (4-5 дней):

- Разработка системы цветов с учетом контрастности
- Создание типографической системы
- Проектирование базовых UI-компонентов
- Разработка иконок и визуальных элементов

5.4. Проектирование основных экранов (5-6 дней):

- Создание макетов главной страницы
- Разработка системы навигации
- Проектирование форм и интерактивных элементов
- Создание шаблонов для различных типов контента

5.5. Разработка адаптивных версий (3-4 дня):

- Адаптация дизайна для мобильных устройств
- Проектирование для планшетов
- Учет особенностей различных размеров экрана

5.6. Создание прототипа (3-4 дня):

- Разработка интерактивного прототипа
- Настройка навигации между экранами
- Имитация основных функций

5.7. Тестирование доступности (4-5 дней):

- Проверка с использованием инструментов оценки доступности
- Тестирование с программами экранного доступа
- Проверка навигации с клавиатуры
- Симуляция различных нарушений зрения

5.8. Доработка дизайна (3-4 дня):

- Внесение изменений на основе результатов тестирования
- Улучшение проблемных мест интерфейса
- Финализация дизайна

5.9. Подготовка документации и презентация (2-3 дня)

6. Критерии успешного выполнения

6.1. Разработанный дизайн соответствует стандартам WCAG 2.1 уровня AA

6.2. Интерфейс удобен для использования всеми целевыми группами пользователей

6.3. Созданы альтернативные версии представления информации для разных потребностей

6.4. Все интерактивные элементы доступны при навигации с клавиатуры

6.5. Дизайн адаптирован для различных устройств и размеров экрана

6.6. Документация по проекту:

- Описание принципов доступного дизайна
- Руководство по использованию компонентов
- Результаты тестирования доступности
- Рекомендации по реализации

7. Дополнительные задания (по желанию)

7.1. Разработка режима для пользователей с дислексией (специальные шрифты и оформление)

7.2. Создание версии с упрощенным языком для пользователей с когнитивными особенностями

7.3. Проектирование голосового управления интерфейсом

7.4. Разработка специальных жестов для пользователей с ограниченной моторикой

7.5. Создание руководства по внедрению доступности для разработчиков

15 кейс. "Интерфейс аналитической платформы": Визуализация данных и бизнес-аналитика

Цель проекта: разработать интуитивно понятный пользовательский интерфейс аналитической платформы, позволяющий эффективно работать с данными, создавать отчеты и визуализировать информацию.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПРОЕКТИРОВАНИЕ

1. Общие требования

1.1. Разработать дизайн веб-интерфейса аналитической платформы для бизнес-пользователей, маркетологов и аналитиков.

1.2. Рекомендуемые инструменты:

- Figma или Adobe XD для создания макетов и прототипов
- Miro или FigJam для построения пользовательских сценариев
- Инструменты для создания примеров визуализаций (можно использовать готовые шаблоны)

1.3. Минимальные требования к дизайну:

- Чистый, функциональный интерфейс без визуального шума
- Адаптивность для различных размеров экранов (от 1366px)
- Высокая информационная плотность с сохранением читаемости

2. Функциональные модули для проектирования

2.1. Модуль авторизации и персонализации

2.1.1. Экраны для проектирования:

- Вход в систему
- Восстановление пароля
- Персональные настройки пользователя
- Управление правами доступа

2.1.2. Требования к дизайну:

- Простой процесс входа
- Четкое отображение текущего пользователя
- Настройки персонализации интерфейса

2.2. Главная страница (дашборд)

2.2.1. Элементы для проектирования:

- Общий обзор ключевых метрик
- Персонализированные виджеты
- Уведомления и оповещения
- Быстрый доступ к часто используемым отчетам
- Навигация по разделам платформы

2.2.2. Требования к дизайну:

- Модульная структура с возможностью настройки
- Баланс между информативностью и читаемостью

- Четкая визуальная иерархия информации

2.3. Модуль работы с данными

2.3.1. Экраны для проектирования:

- Каталог доступных источников данных
- Импорт и загрузка данных
- Просмотр и редактирование наборов данных
- Настройка связей между данными

2.3.2. Требования к дизайну:

- Понятное представление структуры данных
- Интуитивные инструменты для работы с таблицами
- Визуальное отображение связей между данными

2.4. Модуль создания визуализаций

2.4.1. Типы визуализаций для проектирования:

- Линейные и столбчатые графики
- Круговые диаграммы
- Тепловые карты
- Точечные диаграммы
- Таблицы с условным форматированием
- Географические карты
- Индикаторы и счетчики

2.4.2. Экраны для проектирования:

- Выбор типа визуализации
- Настройка параметров графика
- Форматирование и стилизация
- Добавление фильтров и взаимодействий

2.4.3. Требования к дизайну:

- Интуитивный интерфейс создания визуализаций
- Предпросмотр изменений в реальном времени
- Понятные элементы управления настройками

2.5. Модуль создания отчетов и дашбордов

2.5.1. Экраны для проектирования:

- Конструктор дашбордов
- Размещение и компоновка визуализаций
- Настройка интерактивности и фильтров
- Планирование и рассылка отчетов

2.5.2. Требования к дизайну:

- Гибкая система компоновки элементов
- Интуитивное перетаскивание и изменение размеров
- Согласованный внешний вид всех элементов

2.6. Модуль аналитических инструментов

2.6.1. Экраны для проектирования:

- Построение прогнозных моделей
- Поиск аномалий и выбросов
- Сегментация и кластеризация
- Корреляционный анализ

2.6.2. Требования к дизайну:

- Пошаговый процесс настройки аналитических моделей
- Понятное представление сложных параметров
- Наглядная визуализация результатов анализа

2.7. Модуль совместной работы

2.7.1. Экраны для проектирования:

- Общий доступ к отчетам и дашбордам
- Комментирование и обсуждение
- История изменений
- Управление правами доступа к отчетам

2.7.2. Требования к дизайну:

- Четкое отображение прав доступа
- Интуитивные инструменты для комментирования
- Понятная система уведомлений

3. Требования к пользовательскому интерфейсу

3.1. Общие принципы дизайна:

- Минималистичный и функциональный стиль
- Акцент на содержании и данных
- Последовательность и предсказуемость интерфейса
- Эффективное использование пространства экрана

3.2. Цветовая схема:

- Нейтральная основная палитра
- Функциональное использование цвета для визуализаций
- Семантическое использование цветов (зеленый - рост, красный - падение)
- Достаточный контраст для длительной работы

3.3. Типографика:

- Четкая иерархия текста
- Компактные, но читаемые шрифты
- Эффективное использование жирности и размера для выделения

3.4. Навигация:

- Многоуровневая структура с понятной иерархией
- Быстрый доступ к часто используемым функциям
- Хлебные крошки для ориентации в сложной структуре
- Поиск по всей платформе

4. Этапы проектирования (для команды студентов)

4.1. Исследование и анализ (3-4 дня):

- Изучение существующих аналитических платформ (Tableau, Power BI, Google Data Studio)
 - Анализ потребностей различных типов пользователей
 - Определение ключевых сценариев использования
 - Создание портретов пользователей (персон)
 - Сформулировать требования к продукту
 - Создать модели (в Archimate, bpmn, uml) – по желанию
- 4.2. Проектирование информационной архитектуры (2-3 дня):
- Разработка структуры платформы
 - Определение основных разделов и их взаимосвязей
 - Создание карты пользовательских путей
 - Планирование навигации
- 4.3. Создание вайрфреймов (4-5 дней):
- Разработка низкодетализированных макетов основных экранов
 - Проработка расположения элементов управления
 - Определение взаимодействий между экранами
- 4.4. Разработка дизайн-концепции (3-4 дня):
- Определение визуального стиля
 - Создание мудборда (коллекции визуальных референсов)
 - Проработка цветовой схемы и типографики
 - Дизайн ключевых экранов
- 4.5. Проектирование визуализаций данных (4-5 дней):
- Разработка шаблонов для различных типов графиков и диаграмм
 - Создание системы цветового кодирования данных
 - Проектирование интерфейса настройки визуализаций
 - Разработка примеров дашбордов с разными типами данных
- 4.6. Детальное проектирование интерфейса (5-6 дней):
- Создание всех необходимых экранов
 - Проработка элементов управления для разных функций
 - Разработка иконок и визуальных элементов
 - Создание светлой и темной темы
- 4.7. Прототипирование (3-4 дня):
- Создание интерактивного прототипа
 - Настройка переходов между экранами
 - Имитация работы с данными и создания отчетов
- 4.8. Пользовательское тестирование (2-3 дня):
- Подготовка сценариев тестирования
 - Проведение тестирования с потенциальными пользователями
 - Сбор и анализ обратной связи
- 4.9. Доработка дизайна (3-4 дня):
- Внесение изменений на основе результатов тестирования
 - Улучшение проблемных мест интерфейса
 - Финализация дизайна

4.10. Подготовка презентации проекта (2 дня)

5. Критерии успешного выполнения

5.1. Полный набор спроектированных экранов для всех основных функций платформы

5.2. Логичная и понятная информационная архитектура

5.3. Эффективные и информативные визуализации данных

5.4. Удобный интерфейс создания отчетов и дашбордов

5.5. Интерактивный прототип, демонстрирующий основные пользовательские сценарии

5.6. Документация по проекту:

- Описание концепции интерфейса
- Пользовательские сценарии
- Руководство по стилю (цвета, шрифты, компоненты)
- Рекомендации по визуализации различных типов данных

6. Дополнительные задания (по желанию)

6.1. Разработка мобильной версии для просмотра отчетов

6.2. Создание режима презентации для дашбордов

6.3. Проектирование системы оповещений и уведомлений

6.4. Разработка интерфейса для настройки сложных аналитических моделей

6.5. Создание интерактивной обучающей системы для новых пользователей

IV. DevOps

16 кейс. "Создание системы автоматического резервного копирования для малого бизнеса": Защита данных предприятия

Цель проекта: разработать и внедрить простую, но надежную систему автоматического резервного копирования для малого бизнеса, обеспечивающую защиту критически важных данных.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПРОЕКТИРОВАНИЕ

1. Общие требования

1.1. Разработать систему резервного копирования для малого бизнеса (до 20 рабочих станций и 1-2 сервера).

1.2. Рекомендуемые инструменты:

- Bash или PowerShell для написания скриптов
- Rsync, Rclone или аналогичные утилиты для копирования
- Cron (Linux) или Планировщик заданий (Windows) для автоматизации
- Git для контроля версий скриптов

1.3. Минимальные требования к системе:

- Ежедневное инкрементальное резервное копирование
- Еженедельное полное резервное копирование
- Хранение резервных копий локально и в облачном хранилище
- Базовая система оповещений об ошибках

2. Функциональные модули для разработки

2.1. Модуль сбора данных

2.1.1. Функции для реализации:

- Определение критически важных данных для резервного копирования
- Создание списка файлов и директорий для копирования
- Определение приоритетов и частоты копирования

2.1.2. Требования к реализации:

- Конфигурационный файл с настраиваемыми параметрами
- Возможность исключения определенных типов файлов
- Поддержка шаблонов для выбора файлов

2.2. Модуль резервного копирования

2.2.1. Функции для реализации:

- Создание полных резервных копий
- Создание инкрементальных резервных копий
- Сжатие данных для экономии места
- Шифрование резервных копий (опционально)

2.2.2. Требования к реализации:

- Минимальное влияние на производительность системы

- Корректная обработка открытых файлов
- Логирование процесса копирования

2.3. Модуль хранения и ротации

2.3.1. Функции для реализации:

- Локальное хранение резервных копий
- Синхронизация с облачным хранилищем
- Ротация старых резервных копий
- Управление пространством хранения

2.3.2. Требования к реализации:

- Настраиваемые политики хранения (сколько копий хранить)
- Автоматическое удаление устаревших копий
- Проверка доступного пространства перед копированием

2.4. Модуль мониторинга и оповещений

2.4.1. Функции для реализации:

- Проверка успешности резервного копирования
- Отправка уведомлений о результатах
- Базовая статистика использования хранилища
- Обнаружение проблем с резервными копиями

2.4.2. Требования к реализации:

- Отправка email-уведомлений администратору
- Создание журнала событий резервного копирования
- Простой отчет о состоянии системы

2.5. Модуль восстановления

2.5.1. Функции для реализации:

- Восстановление отдельных файлов
- Восстановление полной резервной копии
- Проверка целостности резервных копий

2.5.2. Требования к реализации:

- Простой интерфейс для восстановления
- Возможность выбора версии для восстановления
- Тестовое восстановление для проверки работоспособности

3. Технические требования

3.1. Требования к скриптам:

- Понятный и хорошо документированный код
- Обработка ошибок и исключительных ситуаций
- Возможность запуска в автоматическом и ручном режиме
- Параметризация через конфигурационные файлы

3.2. Требования к производительности:

- Минимальное потребление ресурсов во время работы
- Возможность ограничения скорости передачи данных

- Запуск в нерабочее время для минимизации влияния на бизнес-процессы

3.3. Требования к безопасности:

- Безопасное хранение учетных данных
- Базовое шифрование резервных копий
- Контроль доступа к резервным копиям

3.4. Требования к документации:

- Руководство по установке и настройке
- Инструкция по эксплуатации
- Процедуры восстановления данных

4. Этапы разработки (для команды студентов)

4.1. Анализ и планирование (2-3 дня):

- Определение требований к системе резервного копирования
- Выбор инструментов и технологий
- Создание архитектуры системы
- Распределение задач между членами команды
- Сформулировать требования к продукту
- Создать модели (в Archimate, bpmn, uml) – по желанию

4.2. Настройка среды разработки (1-2 дня):

- Установка необходимого программного обеспечения
- Настройка репозитория Git
- Создание тестовой среды

4.3. Разработка модуля сбора данных (2-3 дня):

- Создание конфигурационного файла
- Написание скрипта для определения файлов для копирования
- Тестирование на различных наборах данных

4.4. Разработка модуля резервного копирования (3-4 дня):

- Написание скриптов для полного и инкрементального копирования
- Реализация сжатия и шифрования
- Тестирование процесса копирования

4.5. Разработка модуля хранения и ротации (3-4 дня):

- Настройка локального хранилища
- Интеграция с облачным хранилищем (например, AWS S3, Google Drive)
- Реализация политик ротации резервных копий

4.6. Разработка модуля мониторинга и оповещений (2-3 дня):

- Создание системы логирования
- Реализация email-уведомлений
- Разработка простой статистики

4.7. Разработка модуля восстановления (3-4 дня):

- Создание скриптов для восстановления данных
- Разработка простого интерфейса для восстановления
- Тестирование процесса восстановления

4.8. Интеграция и тестирование (3-4 дня):

- Объединение всех модулей
- Комплексное тестирование системы
- Исправление обнаруженных ошибок

4.9. Документирование и подготовка к развертыванию (2-3 дня):

- Создание руководства по установке и настройке
- Написание инструкции по эксплуатации
- Подготовка презентации проекта

5. Критерии успешного выполнения

5.1. Работающая система резервного копирования, выполняющая все базовые функции

5.2. Автоматическое выполнение резервного копирования по расписанию

5.3. Успешное хранение копий локально и в облачном хранилище

5.4. Возможность восстановления данных из резервных копий

5.5. Система оповещений, информирующая о результатах копирования

5.6. Документация по проекту:

- Схема архитектуры системы
- Руководство по установке и настройке
- Инструкция по использованию
- Процедуры восстановления данных

6. Дополнительные задания (по желанию)

6.1. Создание простого веб-интерфейса для управления системой

6.2. Реализация дедупликации данных для экономии места

6.3. Настройка репликации резервных копий на несколько хранилищ

6.4. Интеграция с системой мониторинга (например, Zabbix, Nagios)

6.5. Реализация проверки целостности резервных копий

17 кейс. "Создание автоматизированного процесса развертывания простого веб-приложения": Основы CI/CD

Цель проекта: разработать и внедрить простой, но функциональный процесс непрерывной интеграции и доставки (CI/CD) для автоматизированного тестирования и развертывания веб-приложения.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПРОЕКТИРОВАНИЕ

1. Общие требования

1.1. Разработать автоматизированный процесс для тестирования и развертывания простого веб-приложения (статический сайт или простое приложение на Node.js/Python/PHP).

1.2. Рекомендуемые инструменты:

- Git для контроля версий
- GitHub Actions, GitLab CI/CD или Jenkins для автоматизации
- Docker для контейнеризации приложения
- Bash/PowerShell для написания скриптов

1.3. Минимальные требования к процессу:

- Автоматический запуск тестов при отправке изменений в репозиторий
- Автоматическое развертывание приложения при успешном прохождении тестов
- Базовый мониторинг состояния приложения после развертывания

2. Функциональные модули для разработки

2.1. Модуль контроля версий

2.1.1. Функции для реализации:

- Настройка репозитория Git
- Определение стратегии ветвления (Git Flow или упрощенная версия)
- Настройка защиты основных веток

2.1.2. Требования к реализации:

- Четкая структура репозитория
- Правила именования веток и коммитов
- Шаблоны для pull/merge request

2.2. Модуль непрерывной интеграции (CI)

2.2.1. Функции для реализации:

- Автоматический запуск проверок при push или pull request
- Линтинг кода
- Запуск модульных тестов
- Базовая статическая проверка безопасности

2.2.2. Требования к реализации:

- Конфигурация CI в виде кода (YAML или аналог)
- Быстрое выполнение тестов (до 5 минут)
- Понятные отчеты о результатах проверок

2.3. Модуль сборки приложения

2.3.1. Функции для реализации:

- Создание артефакта сборки (build)
- Контейнеризация приложения с Docker
- Версионирование сборок

2.3.2. Требования к реализации:

- Оптимизированный Dockerfile
- Кэширование зависимостей для ускорения сборки
- Минимальный размер итогового образа

2.4. Модуль непрерывной доставки (CD)

2.4.1. Функции для реализации:

- Автоматическое развертывание в тестовую среду
- Ручное подтверждение для развертывания в продакшн
- Откат к предыдущей версии при необходимости

2.4.2. Требования к реализации:

- Настройка тестовой и продакшн сред
- Стратегия развертывания без простоя (zero-downtime)
- Сохранение истории развертываний

2.5. Модуль мониторинга и оповещений

2.5.1. Функции для реализации:

- Проверка доступности приложения после развертывания
- Отправка уведомлений о результатах развертывания
- Базовый мониторинг работоспособности

2.5.2. Требования к реализации:

- Простые проверки здоровья (health checks)
- Email или Slack уведомления
- Логирование процесса развертывания

3. Технические требования

3.1. Требования к инфраструктуре:

- Использование бесплатных облачных ресурсов (GitHub Pages, Netlify, Heroku, AWS Free Tier)
- Минимальные требования к серверам
- Возможность локального тестирования процесса

3.2. Требования к безопасности:

- Безопасное хранение секретов и учетных данных
- Базовое сканирование уязвимостей
- Ограничение доступа к средам развертывания

3.3. Требования к документации:

- Схема CI/CD процесса

- Инструкция по настройке и использованию
- Руководство по устранению типичных проблем

3.4. Требования к масштабируемости:

- Возможность добавления новых этапов в процесс
- Поддержка параллельного выполнения задач
- Возможность расширения для нескольких приложений

4. Этапы разработки (для команды студентов)

4.1. Анализ и планирование (2-3 дня):

- Выбор простого веб-приложения для развертывания
- Определение инструментов и технологий
- Создание схемы CI/CD процесса
- Распределение задач между членами команды
- Сформулировать требования к продукту
- Создать модели (в Archimate, bpmn, uml) – по желанию

4.2. Настройка инфраструктуры (2-3 дня):

- Создание репозитория Git
- Настройка базовой структуры проекта
- Подготовка тестовой и продакшн сред
- Настройка доступов и учетных записей

4.3. Разработка модуля контроля версий (1-2 дня):

- Определение стратегии ветвления
- Настройка защиты веток
- Создание шаблонов для pull request

4.4. Разработка модуля непрерывной интеграции (3-4 дня):

- Настройка автоматических проверок
- Написание базовых тестов
- Настройка линтинга и статического анализа
- Тестирование процесса интеграции

4.5. Разработка модуля сборки приложения (3-4 дня):

- Создание Dockerfile
- Настройка процесса сборки
- Оптимизация образа
- Тестирование сборки в различных условиях

4.6. Разработка модуля непрерывной доставки (3-4 дня):

- Настройка автоматического развертывания
- Создание скриптов для развертывания
- Реализация стратегии развертывания без простоя
- Настройка процедуры отката

4.7. Разработка модуля мониторинга (2-3 дня):

- Создание простых проверок здоровья
- Настройка уведомлений
- Реализация базового мониторинга

4.8. Интеграция и тестирование (3-4 дня):

- Объединение всех модулей
- Комплексное тестирование процесса
- Исправление обнаруженных ошибок
- Оптимизация процесса

4.9. Документирование и подготовка к презентации (2-3 дня):

- Создание документации
- Подготовка демонстрации процесса
- Написание инструкций для пользователей

5. Критерии успешного выполнения

5.1. Работающий CI/CD процесс, автоматически запускающийся при изменениях в репозитории

5.2. Успешное прохождение всех этапов: от коммита до развертывания

5.3. Корректная работа приложения после автоматического развертывания

5.4. Система уведомлений о результатах процесса

5.5. Возможность отката к предыдущей версии

5.6. Документация по проекту:

- Схема CI/CD процесса
- Инструкция по настройке и использованию
- Руководство по устранению типичных проблем

6. Дополнительные задания (по желанию)

6.1. Реализация A/B тестирования при развертывании

6.2. Настройка автоматического масштабирования приложения

6.3. Интеграция с системой управления задачами (Jira, Trello)

6.4. Добавление автоматических интеграционных тестов

6.5. Настройка мониторинга производительности приложения

18 кейс. "Разработка системы мониторинга и автоматического восстановления микросервисов": Обеспечение надежности приложений

Цель проекта: создать простую, но эффективную систему мониторинга и автоматического восстановления для набора микросервисов, обеспечивающую стабильную работу приложения.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПРОЕКТИРОВАНИЕ

1. Общие требования

1.1. Разработать систему мониторинга и автоматического восстановления для простого приложения, состоящего из 3-4 микросервисов.

1.2. Рекомендуемые инструменты:

- Prometheus для сбора метрик
- Grafana для визуализации
- Docker и Docker Compose для контейнеризации
- Bash/Python для написания скриптов автоматического восстановления

1.3. Минимальные требования к системе:

- Мониторинг доступности и базовых метрик микросервисов
- Автоматическое перезапуск сервисов при сбоях
- Система оповещений о проблемах
- Визуализация состояния системы

2. Функциональные модули для разработки

2.1. Модуль тестового приложения

2.1.1. Функции для реализации:

- Простой API-сервис (на Node.js, Python или Go)
- Сервис базы данных (например, MongoDB или PostgreSQL)
- Сервис кэширования (Redis)
- Простой фронтенд для демонстрации работы

2.1.2. Требования к реализации:

- Контейнеризация всех компонентов
- Простая архитектура взаимодействия
- Возможность имитации сбоев для тестирования

2.2. Модуль сбора метрик

2.2.1. Функции для реализации:

- Сбор базовых метрик (CPU, память, диск)
- Мониторинг доступности сервисов (health checks)
- Отслеживание времени отклика API
- Подсчет ошибок и исключений

2.2.2. Требования к реализации:

- Настройка Prometheus для сбора метрик

- Минимальное влияние на производительность сервисов
- Хранение метрик с подходящим разрешением и периодом хранения

2.3. Модуль визуализации

2.3.1. Функции для реализации:

- Создание информационных панелей (дашбордов)
- Визуализация ключевых метрик
- Отображение состояния сервисов
- Графики использования ресурсов

2.3.2. Требования к реализации:

- Настройка Grafana для визуализации
- Создание понятных и информативных дашбордов
- Разные уровни детализации (обзорный и детальный)

2.4. Модуль оповещений

2.4.1. Функции для реализации:

- Настройка правил срабатывания оповещений
- Отправка уведомлений по различным каналам (email, Slack)
- Группировка и фильтрация оповещений
- Отслеживание статуса инцидентов

2.4.2. Требования к реализации:

- Использование AlertManager для управления оповещениями
- Настройка правил для предотвращения шквала оповещений
- Информативные сообщения с рекомендациями по устранению

2.5. Модуль автоматического восстановления

2.5.1. Функции для реализации:

- Автоматический перезапуск неработающих сервисов
- Проверка состояния после восстановления
- Ограничение количества попыток восстановления
- Эскалация проблем при невозможности восстановления

2.5.2. Требования к реализации:

- Скрипты для автоматического восстановления
- Логирование всех действий по восстановлению
- Безопасные стратегии восстановления

3. Технические требования

3.1. Требования к инфраструктуре:

- Возможность запуска на локальной машине для разработки
- Минимальные требования к ресурсам
- Использование контейнеризации для всех компонентов

3.2. Требования к мониторингу:

- Минимальная задержка между сбором метрик (не более 15 секунд)
- Хранение метрик не менее 7 дней

- Возможность агрегации данных для долгосрочного анализа

3.3. Требования к оповещениям:

- Минимальное время реакции на критические проблемы
- Предотвращение ложных срабатываний
- Различные уровни важности оповещений

3.4. Требования к восстановлению:

- Автоматическое восстановление для известных типов сбоев
- Максимальное время восстановления не более 1 минуты
- Корректная обработка зависимостей между сервисами

4. Этапы разработки (для команды студентов)

4.1. Анализ и планирование (2-3 дня):

- Определение архитектуры тестового приложения
- Выбор метрик для мониторинга
- Планирование стратегий восстановления
- Распределение задач между членами команды
- Сформулировать требования к продукту
- Создать модели (в Archimate, bpmn, uml) – по желанию

4.2. Разработка тестового приложения (3-4 дня):

- Создание простых микросервисов
- Настройка взаимодействия между ними
- Контейнеризация компонентов
- Добавление эндпоинтов для проверки здоровья (health checks)

4.3. Настройка системы мониторинга (3-4 дня):

- Установка и настройка Prometheus
- Настройка экспортеров метрик для сервисов
- Настройка сбора базовых метрик
- Тестирование сбора данных

4.4. Разработка визуализации (2-3 дня):

- Установка и настройка Grafana
- Создание основных дашбордов
- Настройка отображения ключевых метрик
- Создание обзорной панели состояния системы

4.5. Настройка системы оповещений (2-3 дня):

- Установка и настройка AlertManager
- Определение правил срабатывания оповещений
- Настройка каналов отправки уведомлений
- Тестирование системы оповещений

4.6. Разработка системы автоматического восстановления (3-4 дня):

- Создание скриптов для проверки состояния сервисов
- Разработка логики восстановления
- Настройка автоматического запуска
- Тестирование процесса восстановления

4.7. Интеграция компонентов (2-3 дня):

- Объединение всех модулей
- Настройка взаимодействия между мониторингом и восстановлением
- Создание единой системы управления

4.8. Тестирование и отладка (3-4 дня):

- Имитация различных сценариев сбоев
- Проверка работы системы восстановления
- Оптимизация настроек оповещений
- Устранение обнаруженных проблем

4.9. Документирование и подготовка к презентации (2-3 дня):

- Создание документации по системе
- Подготовка демонстрации работы
- Написание инструкций по использованию

5. Критерии успешного выполнения

5.1. Работающее тестовое приложение из нескольких микросервисов

5.2. Функционирующая система мониторинга с визуализацией ключевых метрик

5.3. Настроенная система оповещений с разными каналами доставки

5.4. Реализованная система автоматического восстановления

5.5. Успешное прохождение тестовых сценариев сбоев и восстановления

5.6. Документация по проекту:

- Архитектура системы мониторинга и восстановления
- Описание собираемых метрик и правил оповещений
- Инструкция по настройке и использованию
- Сценарии тестирования и их результаты

6. Дополнительные задания (по желанию)

6.1. Добавление трассировки запросов (distributed tracing)

6.2. Реализация автоматического масштабирования сервисов

6.3. Создание исторической аналитики инцидентов

6.4. Настройка мониторинга бизнес-метрик приложения

6.5. Реализация самовосстанавливающейся инфраструктуры с использованием Kubernetes

19 кейс. "Разработка мультиоблачной платформы для управления микросервисами с системой самовосстановления": Современные облачные технологии

Цель проекта: создать простую мультиоблачную платформу для управления микросервисами, обеспечивающую автоматическое развертывание, мониторинг и самовосстановление приложений в разных облачных средах.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПРОЕКТИРОВАНИЕ

1. Общие требования

1.1. Разработать упрощенную платформу для управления микросервисами в нескольких облачных средах (минимум 2 провайдера).

1.2. Рекомендуемые инструменты:

- Kubernetes для оркестрации контейнеров
- Terraform для управления инфраструктурой
- Prometheus и Grafana для мониторинга
- Helm для упаковки и развертывания приложений
- GitHub Actions или GitLab CI для автоматизации

1.3. Минимальные требования к платформе:

- Управление приложениями в нескольких облачных средах
- Автоматическое развертывание микросервисов
- Базовый мониторинг и система оповещений
- Автоматическое восстановление при сбоях

2. Функциональные модули для разработки

2.1. Модуль управления инфраструктурой

2.1.1. Функции для реализации:

- Создание базовой инфраструктуры в разных облаках (AWS, GCP, Azure)
- Настройка сетевого взаимодействия
- Управление ресурсами (вычислительные, хранение, сеть)

2.1.2. Требования к реализации:

- Использование Terraform для описания инфраструктуры как кода
- Модульная структура кода для повторного использования
- Минимальные требуемые ресурсы для экономии

2.2. Модуль оркестрации контейнеров

2.2.1. Функции для реализации:

- Настройка Kubernetes-кластеров в разных облаках
- Управление развертыванием приложений
- Настройка сетевого взаимодействия между сервисами
- Управление конфигурациями приложений

2.2.2. Требования к реализации:

- Использование Helm для упаковки приложений
- Стандартизированный подход к развертыванию
- Поддержка разных стратегий обновления

2.3. Модуль мониторинга и логирования

2.3.1. Функции для реализации:

- Сбор метрик с микросервисов и инфраструктуры
- Централизованное хранение логов
- Визуализация метрик и состояния системы
- Настройка оповещений о проблемах

2.3.2. Требования к реализации:

- Использование Prometheus для сбора метрик
- Настройка Grafana для визуализации
- Базовая система логирования (например, ELK или Loki)

2.4. Модуль самовосстановления

2.4.1. Функции для реализации:

- Автоматическое обнаружение сбоев сервисов
- Перезапуск неработающих компонентов
- Перераспределение нагрузки при проблемах
- Автоматическое масштабирование при необходимости

2.4.2. Требования к реализации:

- Использование встроенных механизмов Kubernetes
- Настройка проверок работоспособности (liveness/readiness probes)
- Правила автомасштабирования (HPA)

2.5. Модуль управления конфигурациями

2.5.1. Функции для реализации:

- Централизованное хранение конфигураций
- Управление секретами
- Версионирование конфигураций
- Применение конфигураций к разным средам

2.5.2. Требования к реализации:

- Использование ConfigMaps и Secrets в Kubernetes
- Базовая система управления секретами
- Разделение конфигураций по средам (dev, test, prod)

2.6. Модуль управления и интерфейса

2.6.1. Функции для реализации:

- Простой CLI-интерфейс для управления платформой
- Базовые операции (развертывание, обновление, удаление)
- Просмотр состояния приложений и инфраструктуры

2.6.2. Требования к реализации:

- Использование bash/python для создания CLI

- Понятные команды и подсказки
- Обработка ошибок и информативные сообщения

3. Технические требования

3.1. Требования к инфраструктуре:

- Поддержка минимум двух облачных провайдеров
- Минимальные требования к ресурсам для снижения затрат
- Использование бесплатных уровней облачных сервисов где возможно

3.2. Требования к безопасности:

- Базовая настройка сетевой безопасности
- Безопасное хранение секретов
- Минимальные привилегии для компонентов системы

3.3. Требования к отказоустойчивости:

- Автоматическое восстановление после сбоев
- Базовая репликация критических компонентов
- Корректная обработка временных сбоев облачных провайдеров

3.4. Требования к масштабируемости:

- Возможность добавления новых облачных провайдеров
- Поддержка горизонтального масштабирования приложений
- Модульная архитектура для расширения функциональности

4. Этапы разработки (для команды студентов)

4.1. Анализ и планирование (2-3 дня):

- Выбор облачных провайдеров для работы
- Определение архитектуры платформы
- Выбор тестовых приложений для развертывания
- Распределение задач между членами команды
- Сформулировать требования к продукту
- Создать модели (в Archimate, bpmn, uml) – по желанию

4.2. Настройка базовой инфраструктуры (3-4 дня):

- Создание учетных записей в облачных сервисах
- Написание Terraform-кода для базовой инфраструктуры
- Настройка сетевого взаимодействия
- Тестирование создания и удаления ресурсов

4.3. Настройка Kubernetes-кластеров (3-4 дня):

- Развертывание кластеров в разных облаках
- Настройка сетевого взаимодействия между кластерами
- Установка базовых компонентов (ingress, storage)
- Тестирование работоспособности кластеров

4.4. Разработка системы управления приложениями (3-4 дня):

- Создание Helm-чартов для тестовых приложений
- Настройка процесса развертывания
- Разработка стратегий обновления
- Тестирование жизненного цикла приложений

4.5. Настройка системы мониторинга (2-3 дня):

- Установка Prometheus и Grafana
- Настройка сбора метрик с приложений и инфраструктуры
- Создание базовых дашбордов
- Настройка простых оповещений

4.6. Реализация системы самовосстановления (3-4 дня):

- Настройка проверок работоспособности для приложений
- Реализация автоматического перезапуска при сбоях
- Настройка правил автомасштабирования
- Тестирование различных сценариев сбоев

4.7. Разработка системы управления конфигурациями (2-3 дня):

- Создание структуры для хранения конфигураций
- Настройка управления секретами
- Реализация применения конфигураций к разным средам
- Тестирование обновления конфигураций

4.8. Создание интерфейса управления (2-3 дня):

- Разработка CLI-команд для основных операций
- Создание скриптов для автоматизации типовых задач
- Добавление подсказок и документации
- Тестирование удобства использования

4.9. Интеграция и тестирование (3-4 дня):

- Объединение всех компонентов
- Тестирование полного цикла работы с приложениями
- Проверка сценариев восстановления после сбоев
- Исправление обнаруженных проблем

4.10. Документирование и подготовка к презентации (2-3 дня):

- Создание документации по платформе
- Подготовка демонстрации работы
- Написание инструкций по использованию

5. Критерии успешного выполнения

5.1. Работающая платформа, поддерживающая минимум два облачных провайдера

5.2. Возможность развертывания и управления микросервисами через единый интерфейс

5.3. Функционирующая система мониторинга и оповещений

5.4. Демонстрация автоматического восстановления после имитации сбоев

5.5. Успешное развертывание тестовых приложений в разных облачных средах

5.6. Документация по проекту:

- Архитектура платформы
- Инструкция по установке и настройке
- Руководство пользователя

- Примеры типовых операций

6. Дополнительные задания (по желанию)

- 6.1. Реализация простого веб-интерфейса для управления платформой
- 6.2. Добавление системы CI/CD для автоматического развертывания приложений
- 6.3. Реализация стратегии аварийного переключения между облаками
- 6.4. Добавление анализа затрат на облачные ресурсы
- 6.5. Реализация системы управления сетевым трафиком (service mesh)